

Article

Research on the Flexible Job Shop Scheduling Problem with Job Priorities Considering Transportation Time and Setup Time

Chuchu Zheng  and Zhiqiang Xie *

School of Computer Science and Technology, Harbin University of Science and Technology, Harbin 150080, China; zhengchuchu_hrbust@163.com

* Correspondence: xiezhiqiang_hrbust@163.com

Abstract

This paper addresses the flexible job-shop scheduling problem with multiple time factors—namely, transportation time and setup time—as well as job priorities (referred to as FJSP-JPC-TST). An optimization model is established with the objective of minimizing the completion time. Considering the characteristics of the FJSP-JPC-TST, we propose an improved whale optimization algorithm that incorporates multiple strategies. First, a two-layer encoding mechanism based on operations and machines is introduced. To prevent illegal solutions, a priority-based encoding repair mechanism is designed, along with an active scheduling decoding method that fully considers multiple time factors and job priorities. Subsequently, a multi-level sub-population optimization strategy, an adaptive inertia weight, and a cross-population differential evolution strategy are implemented to enhance the optimization efficiency of the algorithm. Finally, extensive simulation experiments demonstrate that the proposed algorithm offers significant advantages and exhibits high reliability in effectively solving such scheduling problems.

Keywords: flexible job shop with job priorities; transportation time; setup time; whale optimization algorithm; shop scheduling

MSC: 90C29; 90B35; 90C59



Academic Editor:

Behzad Djafari-Rouhani

Received: 6 November 2025

Revised: 7 December 2025

Accepted: 9 December 2025

Published: 12 December 2025

Citation: Zheng, C.; Xie, Z. Research on the Flexible Job Shop Scheduling Problem with Job Priorities Considering Transportation Time and Setup Time. *Axioms* **2025**, *14*, 914. <https://doi.org/10.3390/axioms14120914>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The Flexible Job-shop Scheduling Problem (FJSP), first introduced by Brucker and Schlie in 1990 [1], has attracted significant research attention due to its broad industrial applicability [2,3]. A key characteristic of the FJSP is that each operation can be processed by multiple functionally similar machines, with processing times for the same operation potentially varying across different machines [4,5]. The primary objective is to optimize the utilization of job-shop resources through efficient allocation of limited available equipment [6]. In recent years, growing demand for customized, low-volume, high-variety production has imposed stricter requirements on production scheduling, particularly for complex product structures. In the manufacturing of industrial products with intricate designs, final products are often assembled from various intermediate components, which are organized hierarchically according to the Bill of Materials (BOM). The completion of child jobs imposes strict constraints on the start times of their parent jobs. Thus, the process flow forms a structured tree that captures both the sequential constraints of operations and the hierarchical constraints among jobs.

On the other hand, the job-shop production time not only refers to the processing time but also includes auxiliary times such as setup time and transportation time. In actual production, enterprises often need to face challenges such as a large variety of products, complex process flows, and parallel production of multiple models, resulting in frequent cross-device transportation and machine setup. Ignoring these auxiliary times will significantly reduce the feasibility of the scheduling plan [7]. In specific job-shop environments such as steelmaking and welding, not considering the setup and transportation times may even cause serious quality problems [8]. It is necessary to further incorporate transportation and setup times into the constraint system in job shop scheduling research, considering job priorities, to optimize process route design and equipment resource allocation, thereby achieving overall scheduling optimization for flexible job shops.

Existing scheduling models often address job priority, transportation time, or setup time in isolation, lacking an integrated approach that captures their synergistic effects in real production environments. To the best of our knowledge, this work is the first to investigate these three constraints simultaneously within a unified framework. Aiming to minimize the makespan, we propose the Flexible Job-shop Scheduling Problem with Job Priority Constraints, Transportation Time, and Setup Time (FJSP-JPC-TST). Unlike the traditional FJSP, where jobs are independent, the FJSP-JPC-TST model incorporates hierarchical job dependencies derived from product BOMs, along with non-negligible transportation and setup times. This integrated approach not only enhances the practical relevance of the scheduling model but also increases its complexity, ultimately supporting the generation of more feasible and effective scheduling solutions for complex manufacturing systems.

Since the FJSP has been proven to be a strongly NP-hard problem [9], the FJSP-JPC-TST studied in this paper also belongs to the NP-hard problem, and a meta-heuristic algorithm with excellent performance needs to be designed to solve this problem. Therefore, we propose an Improved Whale Optimization Algorithm (IWOA) to solve it. IWOA balances global exploration and local exploitation through different hunting methods, and further incorporates a multi-level sub-population optimization strategy, an adaptive inertia weight, and a cross-population differential evolution strategy for optimization. To effectively combine the model and the algorithm, a two-vector coding scheme is proposed, and a correction mechanism is designed to meet the job priority constraints. Finally, the accurate mapping of the production scenario is achieved through decoding.

The structure of this work is arranged as follows: The second section systematically expounds the current research status in the relevant field; Section 3 describes in detail the problem definition and mathematical model of the FJSP-JPC-TST problem; Section 4 proposes a solution method based on the improved whale optimization algorithm; Section 5 covers experimental design, parameter configuration and comparative analysis, verifying the performance of the algorithm through experiments. Section 6 summarizes the research results of the entire text and looks forward to future research directions.

2. Related Work

In traditional FJSP, it is assumed that jobs are independent of each other. To address the production scheduling problem of complex assembly products, researchers introduced job priority constraints into the machining-assembly shop. Vilcot and Billaut [10] were the first to study the priority relationship where multiple predecessors are allowed between operations, but only a single successor. Birgin et al. [11] defined such problems as the extended Flexible Job-shop Scheduling Problem (eFJSP) and established a mixed-integer programming model to simultaneously handle job priority constraints and operation sequence constraints. Based on the above model, researchers have been committed to developing efficient heuristic solution methods. Zou et al. [12] designed a crossover operator and a

mutation operator based on hierarchical constraints. However, since the encoded solution needs to be decomposed into multiple levels, it leads to extremely complex solutions for actual large-scale problems. Zhu et al. [13] proposed an ant colony algorithm combined with simulation and the Neo4j semantic graph method to handle FJSP with priority constraints. However, the frequent modification of graphical data in the Neo4j database significantly reduces the computational efficiency. Zhu and Zhou [14] adopted a three-vector encoding combined with a binary sorting tree priority correction mechanism and designed a discrete gray wolf algorithm embedded with a cellular structure to solve the problem. Gao et al. [15] proposed an improved genetic algorithm. The algorithm designed a new genetic operator to ensure the legality of the encoding and enhanced the optimization ability based on local search of machines. Xie et al. [16] proposed an improved artificial bee colony algorithm. The algorithm introduced the evaluation of machining structure and adopted strategies such as dynamic perturbation step size, double-chain similarity, and migration time factor to optimize the scheduling scheme. Lei et al. [17] proposed an improved genetic algorithm based on an operation relationship matrix table. The algorithm designed an encoding based on a dynamic relationship matrix to ensure sequence constraints, designed new crossover and mutation operators to ensure feasibility, and developed a matrix table decoding method. Zhu et al. [18] proposed a shuffling cell evolutionary gray wolf optimizer, which handles job priority constraints through three-vector coding and binary sort tree repair mechanisms, and achieves parallel optimization by leveraging the cell neighborhood structure. With the in-depth research, more complex scenarios have been taken into consideration. Zhu et al. [14] proposed an optimization algorithm based on the leadership hierarchy of multi-micro groups for the FJSP-JPC problem with interval gray processing time. This algorithm implements dynamic decision-making for machine allocation and the construction of uncertain gray scheduling schemes by establishing a multi-micro-group leadership hierarchy, dividing multiple independent micro-group systems for parallel search, and designing bidirectional quantitative encoding and heuristic active gray decoding methods. Li et al. [19] proposed a hybrid adaptive differential evolution algorithm for the FJSP-JPC problem that considers outsourcing strategies, aiming to minimize the number of work-weighted overdue days under different priority weights. By introducing a constraint processing mechanism, the satisfaction of outsourcing conditions is ensured.

However, the above-mentioned research, while accounting for job priority constraints, failed to fully consider the synergistic impact of job transportation time and machine setup time on scheduling decisions. In actual production and manufacturing, time-consuming auxiliary links, such as transportation and setup time, are key factors that cannot be ignored. Even though the auxiliary time is often shorter than the processing time, the machine may not be able to start actual processing due to waiting for the job to arrive and the machine to be ready, which affects the efficiency and smoothness of the overall production plan [20,21].

In the job-shop scheduling considering transportation time, Zhang et al. [20] proposed an improved memetic algorithm, constructed an elite library and distinguished the treatment of solutions, performed simulated annealing on the general solutions to improve the quality, and applied mutation to the local optimal solutions to enhance diversity. Jiang et al. [22] considered both transportation time and deteriorating processes and designed an improved animal migration optimization algorithm to solve it. Pal et al. [23] proposed a decentralized scheduling based on a multi-agent system, which dynamically allocates machine resources through an auction mechanism. The scheduling agent optimizes the operation sequence by combining Gray Wolf Optimization with critical path-based neighborhood search. Chen et al. [24] proposed a cooperative co-evolutionary algorithm based on genetic programming, using hyper-heuristics to generate scheduling rules and combining with adaptive local search. He et al. [25] considered dynamic events arising from uncertain-

ties in industrial production, established a rescheduling model with time compensation based on transportation time, and proposed a Monte Carlo tree search-based algorithm to solve it. In the job-shop scheduling considering setup time, Meng et al. [26] designed three mixed integer linear programming models and a constraint programming model. Li and Lei [27] proposed an imperialist competitive algorithm with feedback. Song et al. [28] proposed a two-stage heuristic algorithm for the distributed assembly permutation flow job-shop scheduling with setup time. Korhan et al. [29] proposed an adaptive evolutionary strategy algorithm for the distributed permutation flow-shop scheduling problem. Chen et al. [30] proposed an evolutionary multitasking optimization framework that adaptively adjusts the knowledge transfer ratio via dynamic migration rates and employs a genetic programming algorithm as a generative hyper-heuristic to handle dynamic uncertainties in the FJSP, considering transportation and setup times.

In summary, research on the FJSP with job priority constraints (JPC) has predominantly focused on JPC alone, with only a few works extending it to scenarios such as outsourcing strategies or fuzzy processing times. On the other hand, studies on FJSP that consider transportation time (T) or setup time (ST) have mostly extended T or ST to contexts like machine deterioration, distributed scheduling, or uncertain environment. However, no existing work has jointly modeled and optimized job priority constraints together with transportation time and setup time. Integrating job priority constraints (JPC), transportation time (T), and setup time (ST) into a unified scheduling framework not only significantly increases the model's practical relevance but also poses greater challenges to the constraint-handling and search capabilities of solution algorithms. Therefore, we construct the FJSP-JPC-TST model to fill this gap in current research.

As an extension of the FJSP, FJSP-JPC-TST is also NP-hard. Due to the high computational complexity of NP-hard problems, traditional exact algorithms find it difficult to solve them within a reasonable time. Therefore, in recent years, researchers have been committed to developing various meta-heuristic optimization algorithms to effectively handle such problems. These algorithms include genetic algorithm [31,32], particle swarm optimization algorithm [33], ant colony optimization algorithm [34], gray wolf optimization algorithm [35], immune algorithm [36], Harris hawk optimization algorithm [37], memetic algorithm [38], osprey optimization algorithm [39], and sparrow optimization algorithm [40], etc. The Whale Optimization Algorithm (WOA) was proposed by Mirjalili and Lewis [41]. This algorithm simulates the search behavior of whale groups and has the characteristics of simple structure, fast convergence, and strong robustness. It can balance the exploration and exploitation stages of the search space to achieve optimal convergence, and has been practically applied in fields such as feature selection [42], image retrieval [43], robot path planning [44], automatic text summarization [45], and engineering design [46]. As a typical bionic intelligent algorithm, WOA has inherent defects such as being prone to falling into local optima and having a slow convergence speed during the iterative process. For this reason, based on the design of three targeted improvement strategies, we propose an Improved Whale Optimization Algorithm (IWOA) with stronger search ability.

3. Problem Description and Mathematical Model

3.1. Problem Description

In the flexible job shop scheduling scenario, job priorities, transportation time, and setup time need to be clearly defined. Transportation time refers to the time required for a job to transfer between different processing machines; its calculation rule depends on the operation sequence: If the operation is the first operation of a job and the job consists of multiple sub-jobs, the transportation time is the maximum of the times required for each sub-job to independently transfer to the target processing machine. If it is not

the first operation, it refers to the time required to transfer the job from the processing machine of the previous operations to the processing machine of the current operations. Setup time refers to the time required for the machine to perform the preparatory work necessary to meet the processing requirements before starting a new processing task. The specific operations include fixture replacement and equipment parameter reset. Optimizing transportation time can significantly reduce non-productive waiting, and optimizing setup time can enhance the production line's adaptability to multi-variety and small-batch tasks.

The FJSP-JPC-TST model, considering setup time and transportation time, is described as follows. Product P is decomposed into a set $J = \{J_1, J_2, \dots, J_n\}$ containing n jobs according to its tree-shaped bill of materials (BOM). Hierarchical constraint relationships are formed among the jobs through the tree-shaped processing technology structure. In the tree structure, the processing of a parent-node job must wait for the completion of all its child-node jobs. It is defined that a child-node job is a predecessor job of the parent-node job, and the parent node is the successor of the child node. Each job $J_i \in J$ contains a set $O_i = \{O_{i,1}, O_{i,2}, \dots, O_{i,o_i}\}$ of o_i operations that need to be processed sequentially. Among them, operation $O_{i,j+1}$ must start processing only after its predecessor operation $O_{i,j}$ is completed ($j \in \{1, 2, \dots, o_i - 1\}$). The set of available processing equipment is $M = \{M_1, M_2, \dots, M_m\}$. Each operation $O_{i,j}$ needs to select a machine $M_k \in CM_{i,j}$ from the candidate machine set $CM_{i,j} \subseteq M$ for processing, and its processing time $T_{i,j,k}^{proc}$ depends on the production capacity of the assigned machine. In addition, there are transportation time and setup time during the processing gap of the operation. Machine M_k needs a preparation time before processing operation $O_{i,j}$, denoted as $T_{i,j,k}^{setup}$. When operation $O_{i,j}$ is completed on machine M_k , job J_i needs to be transported to the processing machine M_q of the next operation via AGV or robot. The fixed transportation time between machines M_k and M_q is denoted as $TR_{k,q}^{trans}$. The setup of the machine and the transportation of the job do not interfere with each other and can be carried out simultaneously.

To describe the problem model more specifically, the following assumptions are proposed:

- (1) Each operation can only be processed by one machine, and once processing starts, it must be completed continuously without preemption or interruption allowed.
- (2) Each machine can only process one operation at the same time.
- (3) An operation cannot be interrupted during processing until the operation is completed.
- (4) The processing of a parent job must wait for all its child jobs to be completed.
- (5) There is a sufficient number of AGVs or robots, and all jobs can be immediately transported by any available transportation equipment without AGV competition or waiting time.
- (6) The setup time of an operation on a machine only depends on the machine and the operation and is not affected by other factors.
- (7) Situations such as equipment failures, order insertions, and order cancellations are not considered.

Tables 1 and 2 and Figure 1 show an example problem instance. The processing operations within each dashed box belong to the same job. Multiple operations of the same job need to be executed in sequence, and the processing sequence among jobs needs to follow the hierarchical constraint relationship based on BOM. Taking job J_5 as an example, the set of predecessor jobs of job J_5 is $\{J_1, J_2\}$. Assuming that the first operation $O_{5,1}$ of job J_5 is planned to be processed on machine M_1 , then processing can only start after J_1 completes its last operation $O_{1,3}$ and J_2 completes its last operation $O_{2,2}$, and jobs $\{J_1, J_2\}$ are all transported to machine M_1 , and M_1 also completes the previous processing tasks and the setup time $TR_{5,1,1}^{setup}$ for operation $O_{5,1}$. Table 1 gives the basic processing time and setup time of jobs on machines, separated by “/”, and “-” indicates that this operation

cannot be processed on this machine. Table 2 gives the fixed transportation time matrix between machines, with the numbers representing the transportation time.

Table 1. Data model of Product P.

Jobs	Operations	M_1	M_2	M_3	M_4
J_1	$O_{1,1}$	12/4	-	-	-
	$O_{1,2}$	-	8/2	2/2	13/4
	$O_{1,3}$	6/2	-	20/6	-
J_2	$O_{2,1}$	-	17/5	9/3	13/4
	$O_{2,2}$	-	15/4	-	20/6
J_3	$O_{3,1}$	-	19/6	-	-
	$O_{3,2}$	-	-	20/6	4/2
J_4	$O_{4,1}$	-	-	5/2	16/5
	$O_{4,2}$	12/4	-	14/4	-
	$O_{4,3}$	-	-	14/4	-
J_5	$O_{5,1}$	-	-	2/2	19/6
	$O_{5,2}$	-	16/5	1/2	-
J_6	$O_{6,1}$	-	17/5	-	6/2
J_7	$O_{7,1}$	6/2	15/4	-	18/5

Table 2. Transportation Time between Machines.

	M_1	M_2	M_3	M_4
M_1	0	1	1	2
M_2	1	0	1	1
M_3	1	1	0	1
M_4	2	1	1	0

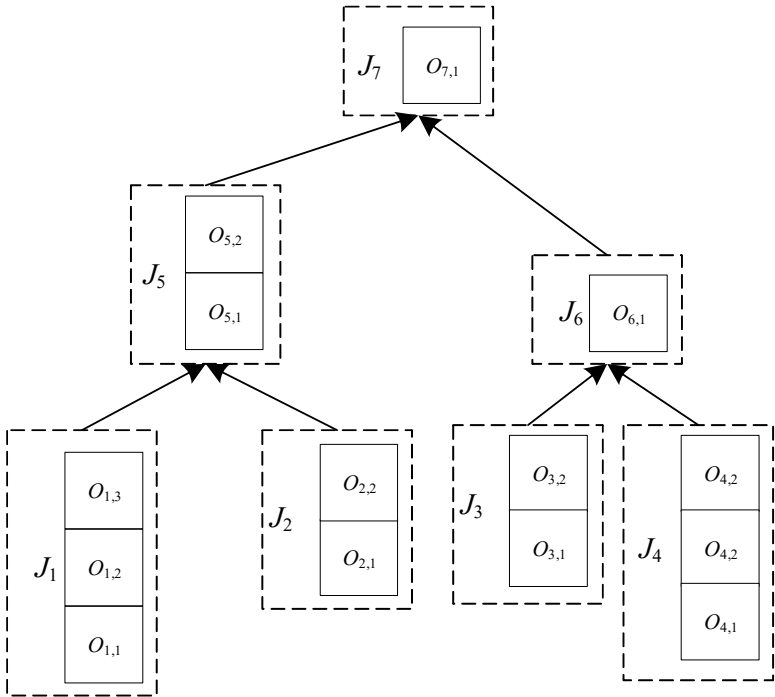


Figure 1. The processing technology tree of Product P (An arrow denotes the precedence constraint between jobs, where the job at the arrow tail must be processed before the job at the arrowhead).

The scheduling scheme Gantt charts of product P are presented in Figure 2. In the figure, processing time is represented by blank bars, with numbers indicating the job and operation numbers. Job transportation time is represented by diagonally striped bars, with numbers identifying destination machine codes. For example, “ $T_{2,1}$ ” indicates the job is being transported from the current machine M_1 to the target machine M_2 for its next operation. Setup time is represented by gray-shaded bars.

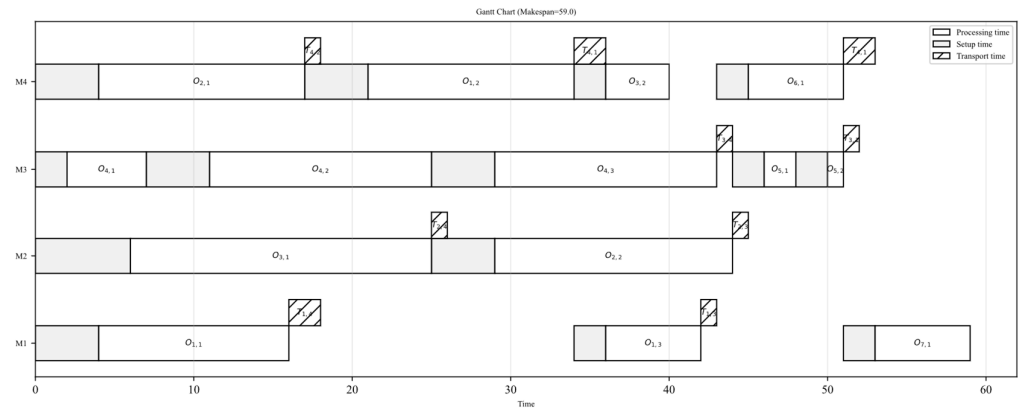


Figure 2. Gantt chart for the scheduling of Product P (Blank bars represent processing time, diagonally striped bars represent transportation time, and gray-shaded bars represent setup time).

3.2. Mathematical Model

The symbol definitions used in the model are shown in Table 3.

Table 3. Symbol definitions.

Symbol Type	Symbol	Symbol Description
Parameter	J	Set of all jobs
	J_i	The i -th job, $i = 1, 2, \dots, n$
	$O_{i,j}$	The j -th operation of job J_i , $j = 1, 2, \dots, o_i$
	M	Set of all machines
	M_k	The k -th machine, $k = 1, 2, \dots, m$
	P_i	Set of predecessor jobs of job J_i
	$\mathcal{M}$	A sufficiently large constant
	jn	Total number of jobs in the product
	o_i	Number of operations in job J_i
	mn	Total number of machines
	$a_{i,j,k}$	1 if machine M_k is a candidate machine for operation $O_{i,j}$, otherwise 0
	$T_{i,j,k}^{proc}$	Processing time of operation $O_{i,j}$ on machine M_k
	$T_{i,j,k}^{setup}$	Setup time of operation $O_{i,j}$ on machine M_k
	$T_{k,q}^{trans}$	Transportation time between machines M_k and M_q
Variable	$S_{i,j}$	Start time of operation $O_{i,j}$
	$C_{i,j}$	Completion time of operation $O_{i,j}$
	C_i	Completion time of job J_i
	$x_{i,j,k}$	1 if $O_{i,j}$ is processed on machine M_k , otherwise 0
	$y_{i,j,i',j',k}$	$y_{i,j,i',j',k} = 1$ when operation $O_{i,j}$ is processed on machine M_k before another operation $O_{i',j'}$, otherwise 0
	$z_{i,j,k',k}$	$z_{i,j,k',k} = 1$ when operation $O_{i,j-1}$ is processed on machine $M_{k'}$ and $O_{i,j}$ is processed on another machine M_k , otherwise 0
	$v_{i,i',k',k}$	$v_{i,i',k',k} = 1$ when the last operation of sub-job J_i is processed on $M_{k'}$ and the first operation of the parent job J_i is processed on another machine M_k , otherwise 0

The research problem in this paper is an extension of the classical NP-hard problem. Its core lies in determining the processing sequence to minimize the makespan. The makespan is a key performance indicator for production scheduling problems, ensuring the production cycle and on-time delivery of enterprises. The objective function and related constraints of the FJSP-JPC-TST model in this paper are defined as follows:

The objective aims to minimize the latest completion time among all jobs, i.e., the global makespan. Define the global makespan as $C_{max} = \max_{1 \leq i \leq n} c_i$. The objective is to minimize C_{max} .

$$f = \min(C_{max}) = \min(\max_{1 \leq i \leq n} c_i) \quad (1)$$

The constraints are grouped into the following categories:

(1) Non-negativity and variable definitions:

$$s_{i,j} \geq 0, \quad \forall i = 1, \dots, jn, j = 1, \dots, o_i \quad (2)$$

$$c_{i,j} = s_{i,j} + \sum_{k \in M} T_{i,j,k}^{proc} x_{i,j,k}, \quad \forall i = 1, \dots, jn, j = 1, \dots, o_i \quad (3)$$

$$c_i = c_{i,j_i}, \quad \forall i = 1, \dots, jn \quad (4)$$

(2) Machine assignment constraints:

$$\sum_{k \in M} x_{i,j,k} = 1, \quad \forall i = 1, \dots, jn, j = 1, \dots, o_i \quad (5)$$

$$x_{i,j,k} \leq a_{i,j,k}, \quad \forall i = 1, \dots, jn, j = 1, \dots, o_i, k \in M \quad (6)$$

(3) Intra-job sequence and transportation constraints:

$$s_{i,j} \geq c_{i,j-1} + \sum_{k' \in M} \sum_{k \in M} T_{k',k}^{trans} \cdot z_{i,j,k',k}, \quad \forall i = 1, \dots, jn, j = 2, \dots, n_i \quad (7)$$

$$z_{i,j,k',k} \leq x_{i,j-1,k'}, \quad \forall i = 1, \dots, jn, j = 2, \dots, o_i, k', k \in M \quad (8)$$

$$z_{i,j,k',k} \leq x_{i,j,k}, \quad \forall i = 1, \dots, jn, j = 2, \dots, o_i, k', k \in M \quad (9)$$

$$z_{i,j,k',k} \geq x_{i,j-1,k'} + x_{i,j,k} - 1, \quad \forall i = 1, \dots, jn, j = 2, \dots, o_i, k', k \in M \quad (10)$$

(4) Inter-job precedence (BOM) and transportation constraints:

$$s_{i,1} \geq c_{i',J_{i'}} + \sum_{k' \in M} \sum_{k \in M} T_{k',k}^{trans} \cdot v_{i,i',k',k}, \quad \forall i = 1, \dots, jn : P_i \neq \emptyset, J_{i'} \in P_i \quad (11)$$

$$v_{i,i',k',k} \leq x_{i',J_{i'},k'}, \quad \forall i = 1, \dots, jn : P_i \neq \emptyset, J_{i'} \in P_i, k', k \in M \quad (12)$$

$$v_{i,i',k',k} \leq x_{i,1,k}, \quad \forall i = 1, \dots, jn : P_i \neq \emptyset, J_{i'} \in P_i, k', k \in M \quad (13)$$

$$v_{i,i',k',k} \geq x_{i',J_{i'},k'} + x_{i,1,k} - 1, \quad \forall i = 1, \dots, jn : P_i \neq \emptyset, J_{i'} \in P_i, k', k \in M \quad (14)$$

(5) Machine processing sequence with setup time constraints:

$$y_{i,j,i',j',k} + y_{i',j',i,j,k} \geq x_{i,j,k} + x_{i',j',k} - 1, \quad \forall k \in M, \forall (i,j,i',j') : (i,j) \neq (i',j') \quad (15)$$

$$s_{i',j'} \geq c_{i,j} + T_{i',j',k}^{setup} - \mathcal{M}(1 - y_{i,j,i',j',k}), \quad \forall k \in M, \forall (i,j,i',j') : (i,j) \neq (i',j') \quad (16)$$

$$s_{i,j} \geq c_{i',j'} + T_{i,j,k}^{setup} - \mathcal{M}(1 - y_{i',j',i,j,k}), \quad \forall k \in M, \forall (i,j,i',j') : (i,j) \neq (i',j') \quad (17)$$

Equation (2) indicates that the start time of all operations must be non-negative. Equation (3) calculates the end time of each operation. Equation (4) calculates the end time of each job, and the end time of a job is the end time of its last operation. Equation (5) ensures that each operation can only be completed on one machine. Equation (6) ensures that each operation must be processed by its candidate machine. Equation (7) represents

the operation sequence constraints (within the same job), which define that the start time of each non-first operation in a job must wait for the completion of the previous operation and add the transportation time. Equations (8)–(10) together ensure that the transportation indicator variable $z_{i,j,k',k}$ equals 1 if and only if operation $O_{i,j-1}$ is processed on machine $M_{k'}$ and operation $O_{i,j}$ is assigned to machine M_k . Equation (11) represents the job hierarchy constraints (between jobs), which define that the first operation of the parent job must wait for all child jobs to be completed, and add the transportation time. Equations (12)–(14) activate the transportation variable $v_{i,i',k',k}$ precisely when the last operation of child job $J_{i'}$ is finished on machine $M_{k'}$ and the first operation of parent job J_i is scheduled on machine M_k . Equations (15) and (16) define the machine processing sequence constraints, stipulating that for two adjacent operations processed on the same machine, the processing times cannot overlap, and the subsequent operation needs to wait for the previous one to end and add the machine setup time.

4. Solution Method

We propose an improved whale optimization algorithm (IWOA). First, a designed encoding scheme transforms the actual production job-shop scenario into a mathematical representation, and a repair mechanism based on job priorities is designed to ensure feasibility. Second, a hierarchical sub-population collaboration strategy is established to achieve intensive local optimization within sub-populations and enable the migration of high-quality solutions between them. Concurrently, an adaptive inertia weight and a cross-population differential evolution strategy are introduced to enhance the balance between exploration and exploitation, thereby optimizing the population update process. Finally, the solutions are decoded and restored into an executable production scheduling plan.

4.1. Encoding and Decoding

We adopt a two-vector coding scheme, considering job sequencing and machine allocation. To ensure the feasibility of the solution and meet the job priority constraints, a repair mechanism based on job priority is used.

4.1.1. Encoding

A two-vector encoding scheme is adopted. The first layer is the operation sequence vector (OS), which determines the processing order of job operations. Repeated integers represent different operations belonging to the same job. The second layer is the machine selection vector (MS), which identifies the specific machine chosen from the available equipment set for each corresponding operation. Both encoding layers have the same length, equal to the total number of operations, calculated as $I = \sum_i^n o_i$. Taking the instance of 7 jobs and 4 machines described in Section 3.1 (hereafter referred to as Example 1), the OS vector has a length of 14. As shown in Figure 3, the first element “2” in the OS vector represents operation $O_{2,1}$. According to the corresponding element in the MS vector, this operation is assigned to be processed on machine M_2 , selected from its available alternatives. The second element “2” in OS represents operation $O_{2,2}$, which is assigned to machine M_1 .

The standard Whale Optimization Algorithm (WOA) is primarily designed for continuous optimization problems. To adapt it for solving discrete scheduling problems, specific conversion rules for both machine encoding and operation encoding are introduced. For machine encoding conversion, the values in the machine position vector are transformed into indices pointing to the candidate machine sets for the corresponding operations, thereby determining the final machine assignments. The conversion is defined by Equation (18).

$$u(e) = \text{round}\left(\frac{x(e) + \epsilon}{2\epsilon} \cdot (s(e) - 1) + 1\right), 1 \leq e \leq I \quad (18)$$

In this context, $s(e)$ denotes the number of available machines for the operation corresponding to element e ; $x(e)$ represents the value of element e in the machine encoding layer; $u(e)$ signifies the index within the candidate machine set assigned to this operation. The element values at individual positions are confined to the interval $[-\epsilon, \epsilon]$. Here, ϵ is a normalization constant. Based on empirical tuning, we set $\epsilon = 8$ for our experiments. The experiment found that this value provides a sufficient numerical range to stably map continuous values to discrete machine indexes. Correspondingly, when converting a scheduling plan back into machine encoding, the inverse operation of Equation (18) is applied. A numeric example is provided to illustrate the steps. Suppose $\epsilon = 8$, and for a certain operation, the number of candidate machines is 3, and the continuous value $x = -1.5$. Then, the machine index is calculated as follows: $u = \text{round}((-1.5 + 8)/(2 \times 8) \times (3 - 1) + 1) = 2$. Thus, the second machine in the candidate set is selected.

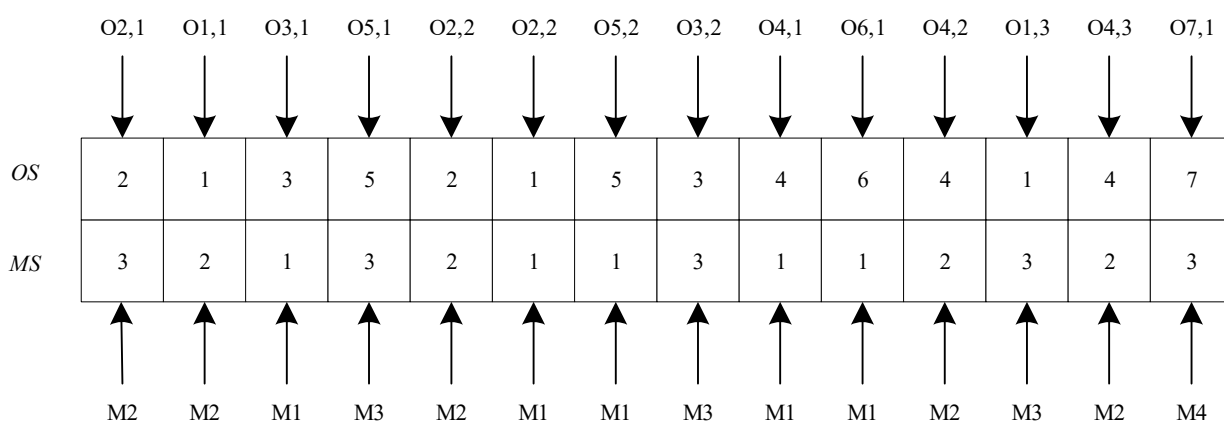


Figure 3. Example of decoding OS and MS into a scheduling solution (In the OS, the numbers represent Job IDs. Their sequence indicates the processing order of operations. The n th occurrence of the same number represents the n th operation of that job. In the MS, the numbers represent Machine IDs assigned to the corresponding operation. The same applies hereafter).

For operation encoding conversion, the Ranked Order Value (ROV) rule is employed. First, the element values within the operation encoding vector are sorted in ascending order, and corresponding ROVs are assigned to each element. Subsequently, based on the sequence of these ROVs, the operations represented by the elements are ordered to generate an operation sequence, as illustrated in Figure 4. Similarly, the reverse process is adopted when converting a scheduling plan back into operation encoding.

To address the prevalence of infeasible candidate solutions that violate the mixed constraints among randomly generated solutions, we design a correction mechanism based on job priorities to rectify these solutions. First, the original operation sequence (OS) and the given job priority tree, which defines the precedence constraints between jobs, are taken as input. Then, a binary search tree is constructed using the first element in the OS sequence as the root node. For each subsequent element, starting from the root node, a recursive comparison based on priority relationships is performed. If the job corresponding to the current element has a higher priority than the job of the comparison node, the element is inserted into the left subtree; otherwise, it is inserted into the right subtree. This recursive process continues downward until an empty position is found for insertion, causing higher-priority jobs to progressively gather toward the lower-left corner of the tree. Finally, an in-order traversal is performed on the constructed binary search tree, visiting nodes in

the order of left subtree, root node, and right subtree, thereby generating a new operation sequence that fully complies with the priority constraints.

Random position	−5.96	4.22	−2.33	−6.54	−3.05	0.54	−4.10	1.47	3.76	5.78	−3.87	−1.56	2.68	6.90
OS	1	1	1	2	2	3	3	4	4	4	5	5	6	7
↓														
ROV ascending order	−6.54	−5.96	−4.10	−3.87	−3.05	−2.33	−1.56	0.54	1.47	2.68	3.76	4.22	5.78	6.90
OS	2	1	3	5	2	1	5	3	4	6	4	1	4	7

Figure 4. Illustration of the ROV rule for OS (Row 2 lists the job IDs for all operations. Each of these IDs is assigned a random value from Row 1. The values are sorted ascendingly to produce the ROV sequence (Row 3), whose order is then mapped back to the job IDs in Row 2 to yield the final operation sequence (Row 4)).

The job priority tree for Example 1 is shown in Figure 5. By applying the aforementioned correction mechanism, the solution presented in Figure 5 is adjusted to obtain an encoding scheme that satisfies the mixed constraints, as demonstrated in Figure 6.

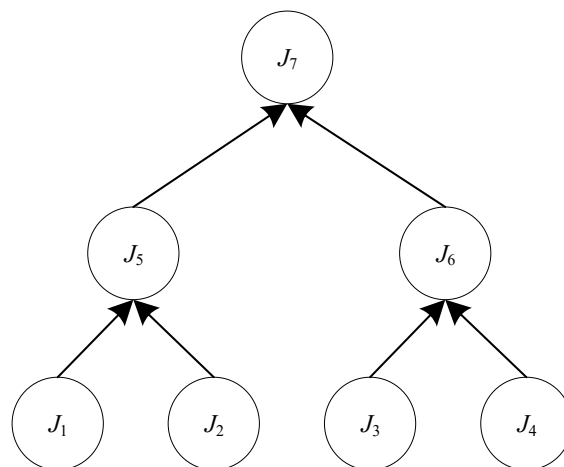


Figure 5. Job priority tree (defined by the BOM structure; an arrow from a child job points to its parent job, indicating that the job at the tail of the arrow must be processed before the job at the head of the arrow can start).

OS	2	1	3	2	1	3	4	4	1	4	5	5	6	7
----	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Figure 6. A repaired OS encoding after applying the job-priority correction mechanism.

4.1.2. Decoding

An active scheduling decoding strategy is adopted. Without violating the mixed priority constraints, the decoding process is not strictly bound by the sequence defined in the operation encoding. Instead, it preferentially selects the arrangement that allows the operation to start processing at the earliest possible time. The specific steps are as follows: Traverse all operations in the OS from left to right. For the current operation $O_{i,j}$ to be

scheduled, read the assigned machine M_k from the MS encoding and calculate its earliest start time $EST_{i,j}$, considering multiple constraints:

- (1) For the non-first operation $O_{i,j}$ ($j \geq 2$) of any job, its start time $EST_{i,j}$ must meet the following combined conditions: First, it is necessary to wait for the previous operation $O_{i,j-1}$ to be completed on machine M_p , and add the transportation time $T_{p,k}^{trans}$ of the job from machine M_p to the current machine M_k ; at the same time, the current machine M_k needs to complete the previous processing task and the setup time $T_{i,j,k}^{setup}$ for operation $O_{i,j}$. Since the transportation process and machine setup can be executed in parallel, the actual value of $EST_{i,j}$ is determined by the maximum of the transportation ready time and the machine ready time, that is:

$$EST_{i,j} = \max(c_{i,j-1} + T_{p,k}^{trans}, F_k + T_{i,j,k}^{setup}) \quad (19)$$

where F_k is the completion time of the last operation processed on machine M_k .

- (2) For the first operation $O_{i,1}$ of any job, if job J_i is a parent job, it must additionally wait for the processing and transportation of all its child jobs to be completed, while still satisfying the machine setup constraint as follows:

$$EST_{i,1} = \max(\max_{i' \in P_i}(c_{i'} + T_{m',k}^{trans}), F_k + T_{i,1,k}^{setup}) \quad (20)$$

where $m_{i'}$ is the processing machine of the child job $J_{i'}$. If job J_i has no predecessor job, then:

$$EST_{i,1} = F_k + T_{i,1,k}^{setup} \quad (21)$$

Subsequently, select the earliest feasible time interval that can accommodate the operation's processing time by identifying the earliest idle window on the timeline of machine M_k where the current operation can be inserted. Specifically, check each idle period of the machine in chronological order from left to right to determine if it can accommodate the processing time of the current operation. Once a suitable period is found, schedule the operation to start at the beginning of that period and dynamically update the machine's timeline. If no existing idle window meets the requirement, append the operation to the end of the machine's task queue. Let the scheduled working time window of machine M_k be $W_k = \{(mst_{k,1}, met_{k,1}), (mst_{k,2}, met_{k,2}), \dots, (mst_{k,n}, met_{k,n})\}$, representing the set of already scheduled operations in chronological order. The idle windows can then be derived as $F_k = \{(0, mst_{k,1}), (et_{k,1}, mst_{k,2}), \dots, (met_{k,n}, \infty)\}$. For each idle window $[A, B]$, if it satisfies:

$$A + T_{i,j,k}^{proc} \leq B \text{ and } A \geq EST_{i,j} \quad (22)$$

Then, schedule $O_{i,j}$ in this window the actual start time of the operation $s_{i,j} = A$. If no window satisfies the conditions, append the operation to the end of the machine's schedule, setting its actual start time $s_{i,j} = \max(met_n, EST_{i,j})$, then calculate the actual end time of the operation $c_{i,j} = A + T_{i,j,k}^{proc}$, update the working time window of machine M_k , and reorder.

$$W_k = W_k \cup \{(s_{i,j}, c_{i,j})\} \quad (23)$$

Iterate through the above process sequentially and record the start and completion times of each operation. Finally, obtain a complete scheduling plan and use the total production makespan as the fitness value.

4.1.3. Population Initialization

An initial population is generated using a random initialization strategy. For the OS encoding, solutions are initialized randomly. For the MS encoding, three distinct initialization methods are employed with equal probability: (1) randomly selecting a machine from the candidate set; (2) selecting the machine with the shortest processing time

for the operation from the candidate set; (3) selecting the machine with the smallest current workload from the candidate set.

4.2. Traditional WOA

The Whale Optimization Algorithm (WOA) simulates the hunting behavior of hump-back whales, where initial solutions are iteratively updated to approach the global optimum. The hunting process consists of three main phases: encircling prey, bubble-net attacking, and searching for prey. An adaptive switching strategy governs the transition between these phases.

- ① Encircling Prey: Individuals move towards the current best solution to narrow the encirclement. The position update formulas are:

$$\vec{X}_i^{t+1} = \vec{X}_{best}^t - \vec{A} \cdot \vec{D}_1 \quad (24)$$

$$\vec{D}_1 = |\vec{C} \cdot \vec{X}_{best}^t - \vec{X}_i^t| \quad (25)$$

$$\vec{A} = 2a\vec{r}_1 - a \quad (26)$$

$$\vec{C} = 2\vec{r}_2 \quad (27)$$

$$a = 2 - 2t/T_{max} \quad (28)$$

Here, $\vec{X}_i^{t+1}$ is the position vector of the individual i at the t -th iteration; $\vec{X}_{best}^t$ is the position vector of the individual with the optimal fitness at the t -th iteration; $\vec{D}_1$ is the encirclement vector. The parameter a is a linear convergence factor, and its value gradually decreases from 2 to 0 as the number of iterations increases. $\vec{r}_1$ and $\vec{r}_2$ are random vectors in the $[0, 1]$ interval, and T_{max} is the maximum number of population updates. $\vec{A}$ and $\vec{C}$ continuously adjust the change of the whale individual's position according to the change of a , enabling the current individual to reach different positions around the best whale individual.

- ② Bubble-net Attacking: After encircling the prey, whales perform a spiral bubble-net attack. They move towards the prey along a logarithmic spiral path while the attack radius decays with iterations. The position update formula is:

$$\vec{X}_i^{t+1} = \vec{X}_{best}^t + e^{bl} \cos(2\pi l) \vec{D}_2 \quad (29)$$

$$\vec{D}_2 = |\vec{X}_{best}^t - \vec{X}_i^t| \quad (30)$$

Here, $e^{bl} \cos(2\pi l)$ controls the whale individual to search for the prey in a logarithmic spiral descent manner. The constant b is a constant used to control the spiral shape, usually set to 1. l is a random number in the $[0, 1]$ interval. $\vec{D}_2$ is the absolute value of the distance between the current individual and the best individual.

- ③ Searching for Prey: To explore new areas, whales randomly select another individual and move towards it. The position update formulas are:

$$\vec{X}_i^{t+1} = \vec{X}_{rand}^t - \vec{A} \cdot \vec{D}_3 \quad (31)$$

$$\vec{D}_3 = |\vec{C} \cdot \vec{X}_{rand}^t - \vec{X}_i^t| \quad (32)$$

Here, $\vec{X}_{rand}^t$ is a randomly selected whale's position at iteration t ; $\vec{D}_3$ is the random search vector.

- ④ **Adaptive Switching Strategy:** This strategy intelligently balances global exploration and local exploitation through the dynamic parameter $\vec{A}$. The absolute value $|\vec{A}|$ exhibits distinct phase characteristics. In early iterations, a larger a often results in $|\vec{A}| \geq 1$, promoting global exploration. Whales are guided toward a randomly selected individual $\vec{X}_{rand}^t$ (Equation (31)), introducing random perturbations that help escape local optima and explore unknown regions, thus maintaining population diversity. As iterations progress and a decays, $|\vec{A}|$ typically falls below 1, switching to local exploitation. Whales then focus on the current best solution $\vec{X}_{best}^t$ and select with equal probability (50%) either the shrinking encirclement (Equation (24)) or the spiral update path (Equation (29)). This dual mechanism enables fine-grained search near the optimum, using spiral paths to avoid local traps, thereby enhancing convergence accuracy and speed.

4.3. Improved IWOA

4.3.1. Multi-Level Sub-Population Optimization Strategy

In nature, most organisms exist in groups. Individuals with similar habits congregate in specific environments, forming smaller sub-populations. Within each sub-population, individuals share similar living characteristics. The “survival of the fittest” co-evolution is achieved through resource competition within populations, while information exchange occurs between them [47].

Inspired by this ecological mechanism, we establish a hierarchical sub-population collaborative optimization strategy. During initialization, the population is randomly divided into several sub-populations of equal size, each operating as an independent evolutionary unit. Different from traditional methods, we further introduce a hierarchical stratification mechanism. The population is evenly stratified into multiple levels based on fitness values. Each sub-population is then constructed through stratified random sampling to ensure it contains an equal number of individuals from different levels. This approach balances the guidance from elite solutions with the stability of global search. Each sub-population evolves autonomously according to the position update rules of the whale optimization algorithm. This distributed strategy enables different sub-populations to deeply explore distinct regions of the solution space, forming local convergence cores.

To avoid local optima, a recombination mechanism is implemented. After each round of evolution, the quality of the solution set is evaluated based on fitness values. If the optimal solution shows significant improvement, the current structure is maintained. Otherwise, the recombination process is initiated: sub-populations are merged into a complete population, and new sub-populations are reconstructed through stratified random sampling. This facilitates information exchange between sub-populations and helps prevent each sub-population from becoming trapped in local optima during optimization. Geographical overlap may exist between sub-populations, leading to repeated searches of the same region by individuals from different sub-populations during iterations. Although such regional overlap might reduce search efficiency, it can accelerate the location of the global optimum when that optimum lies within the overlapping region. Furthermore, as problem dimensionality increases, overlapping searches in multi-dimensional space contribute to obtaining higher-precision solutions.

4.3.2. Adaptive Inertia Weight

As a key parameter in the Whale Optimization Algorithm (WOA), the inertia weight significantly influences the position update process. The core mechanism of its adaptive adjustment lies in adopting a larger weight during the initial algorithm phase to enhance

global exploration capability, and a smaller weight during later iterations to improve local exploitation ability. By adaptively configuring the inertia weight, the algorithm achieves a better balance between exploration and exploitation stages, thereby effectively improving search efficiency and overall performance. Therefore, we design two inertia weight coefficients: the cosine decay inertia weight coefficient w and the exponential inertia weight coefficient v , defined as follows:

$$w(t) = w_{\min} + (w_{\max} - w_{\min}) \times \frac{1 + \cos(\pi \times \frac{t}{T})}{2} \quad (33)$$

$$v(t) = w_{\max} \times \left(\frac{w_{\min}}{w_{\max}} \right)^{\left(\frac{t}{T} \right)^k} \quad (34)$$

Here, $w_{\max}$ and $w_{\min}$ represent the maximum and minimum values of the decaying inertia weight coefficient, respectively, and the decay behavior is governed by the current iteration count t and the maximum iteration number T . The parameter k controls the decay rate of the exponential weight. Finally, the individual position update formulas are modified by incorporating w and v . Table 4 summarizes how $w(t)$ and $v(t)$ are assigned to each update mode of IWOA, and the corresponding modifications to the position-update formulas are given in Equation (35).

$$\vec{X}_i^{t+1} = \begin{cases} w(t) \cdot \vec{X}_{best}^t - \vec{A} \cdot \vec{D}_1 & \text{if } p < 0.5 \text{ and } |\vec{A}| < 1 \\ w(t) \cdot \vec{X}_{best}^t + e^{bl} \cos(2\pi l) \vec{D}_2 & \text{if } p < 0.5 \text{ and } |\vec{A}| \geq 1 \\ v(t) \cdot \vec{X}_{rand}^t - \vec{A} \cdot \vec{D}_3 & \text{if } p \geq 0.5 \end{cases} \quad (35)$$

Table 4. Inertia Weight Usage in IWOA Update Modes.

Update Mode	Condition	Inertia Weight	Corresponding Behavior
Encircling prey	$p < 0.5$ and $ \vec{A} < 1$	$w(t)$	Local exploitation: shrinking towards the current best
Bubble-net attacking	$p < 0.5$ and $ \vec{A} \geq 1$	$w(t)$	Local exploitation: spiral search around the best
Searching for prey	$p \geq 0.5$	$v(t)$	Global exploration: moving towards the random individual

4.3.3. Cross-Population Differential Evolution Strategy

To address the limitation of the Whale Optimization Algorithm (WOA) being prone to local optima, we propose a cross-population differential evolution strategy that integrates the differential evolution (DE) mutation operator with inter-population communication. This strategy operates through two main mechanisms: First, the DE mutation operator is introduced, leveraging the positional differences between the current population's optimal solution and neighboring individuals to drive the population away from local optima. Second, by incorporating elite solution information from other sub-populations, a gradient-like search direction across sub-populations is constructed, thereby expanding the global exploration scope. Additionally, the random perturbation characteristics of the Gaussian function are utilized to introduce multi-directional disturbances when population stagnation is detected.

In the DE algorithm, the mutation process serves as the core mechanism for population evolution. It achieves a balance between global exploration and local exploitation by applying directional perturbations to each dimension of an individual's position. We adopt the "DE/current-to-best/1" mutation operator [48], whose formulation is as follows:

$$\vec{X}_{a,curr}^{\rightarrow t+1} = \begin{cases} \vec{X}_{a,curr}^{\rightarrow t} + F \cdot (\vec{X}_{a,best}^{\rightarrow t} - \vec{X}_{a,curr}^{\rightarrow t}) + F \cdot (\vec{X}_{a,r1}^{\rightarrow t} - \vec{X}_{a,r2}^{\rightarrow t}), & \text{rand} \leq 0 \\ \vec{X}_{a,curr}^{\rightarrow t} + F \cdot (\vec{X}_{a,best}^{\rightarrow t} - \vec{X}_{a,curr}^{\rightarrow t}) + F \cdot (\vec{X}_{b,elite}^{\rightarrow t} - \vec{X}_{b,r3}^{\rightarrow t}), & \text{rand} > 0 \end{cases} \quad (36)$$

Here, $\vec{X}_{a,best}^{\rightarrow t}$, $\vec{X}_{a,curr}^{\rightarrow t}$, $\vec{X}_{a,r1}^{\rightarrow t}$, $\vec{X}_{a,r2}^{\rightarrow t}$ represent the position vectors of the best individual, the current individual *curr* being updated, and random individuals *r1* and *r2* from the current sub-population *sub_a* at iteration *t*, respectively, with $r1 \neq r2 \neq curr$. $\vec{X}_{b,elite}^{\rightarrow t}$, $\vec{X}_{b,r3}^{\rightarrow t}$ represent the position vectors of an arbitrarily selected elite individual and a random individual *r3* from another sub-population *sub_b* at iteration *t*, respectively. *F* is a random scaling factor within [0, 0.5], *rand* is a random number uniformly distributed in [0, 1], and *p* is a predefined probability threshold controlling the operator selection.

The Gaussian function is employed to perturb each dimension of the position vector. The probability density function of the Gaussian distribution is:

$$\mathcal{N}(x | \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right) \quad (37)$$

The DE mutant solution $\vec{X}_{a,curr}^{\rightarrow t+1}$ after adding a Gaussian perturbation is as follows:

$$\vec{X}_{a,curr}^{\rightarrow t+1} = \vec{X}_{a,curr}^{\rightarrow t+1} + \mathcal{N}(0, \sigma^2) \cdot (\vec{U} - \vec{L}) \quad (38)$$

Here, $\sigma = e^{-\lambda(t/T_{\max})}$ is the adaptive standard deviation, λ is a random attenuation coefficient with values between [1, 5], and $\vec{U} - \vec{L}$ is the solution space scale vector. Since the solution space is normalized to $[-\varepsilon, \varepsilon]$, $\vec{U} - \vec{L}$ is a constant vector with each component equal to 2ε . If a component exceeds the interval $[-\varepsilon, \varepsilon]$, after perturbation, a reflection method is used to map it back into the feasible domain. For a value x' , the corrected value x'' is corrected as follows:

$$x'' = \begin{cases} 2\varepsilon - x' & \text{if } x' > \varepsilon, \\ -2\varepsilon - x' & \text{if } x' < -\varepsilon, \\ x' & \text{otherwise.} \end{cases} \quad (39)$$

4.4. Algorithm Steps

Step 1: Parameter Initialization. Set the current number of iterations *iter* = 1, the maximum number of iterations *T_{max}*, population size *NP*, number of sub-populations *n*, and inertia weight coefficients *w_{min}* and *w_{max}*. Calculate the sub-population size as *N_{sub}* = *NP*/*n*.

Step 2: Population Initialization and Repair. Generate the initial population using the encoding scheme. Apply the priority constraint repair mechanism to each individual's Operation Sequence (OS) to ensure compliance with the job hierarchy constraints defined by the Bill of Materials (BOM) tree structure.

Step 3: Fitness Evaluation. Calculate the fitness value for each individual using the active scheduling decoding strategy, where fitness is defined as $1/C_{\max}$. Record the global best solution and its corresponding fitness value.

Step 4: Hierarchical Sub-population Construction. Sort the entire population in ascending order of fitness and stratify it evenly into four tiers: top 25%, second 25%, third 25%, and bottom 25%. Construct each sub-population through stratified random sampling, ensuring an equal number of individuals are selected from each tier to maintain diversity across solution qualities.

Step 5: Sub-population Evolution. For each whale individual within every sub-population, execute the following steps:

Step 5.1: Update search parameters using adaptive inertia weights.

Step 5.2: Designate the individual with the best fitness as the leader and update positions according to the WOA strategy, followed by priority constraint repair.

Step 5.3: Apply the cross-population differential evolution strategy to the top 20% of elite individuals, followed by priority constraint repair. Replace the original solution if the new solution demonstrates improved fitness.

Step 5.4: Merge the updated sub-population with its previous version, then select the top N_{sub} solutions based on fitness to form the new sub-population.

Step 6: Global Best Solution Update. Compare the best individual from each sub-population against the current global best solution. Update the global best solution if a superior candidate is found.

Step 7: Termination Check. If the $iter$ reaches T_{max} , terminate the algorithm and output the complete scheduling plan corresponding to the global best solution. Otherwise, $iter = iter + 1$, proceed to Step 8.

Step 8: Population Reorganization. If the global best solution shows improvement in the current iteration, return to Step 5 to continue sub-population evolution. If no improvement occurs, merge all sub-populations into a unified population and return to Step 4 to reconstruct new sub-populations through re-stratification.

The complete algorithm workflow is illustrated in Figure 7.

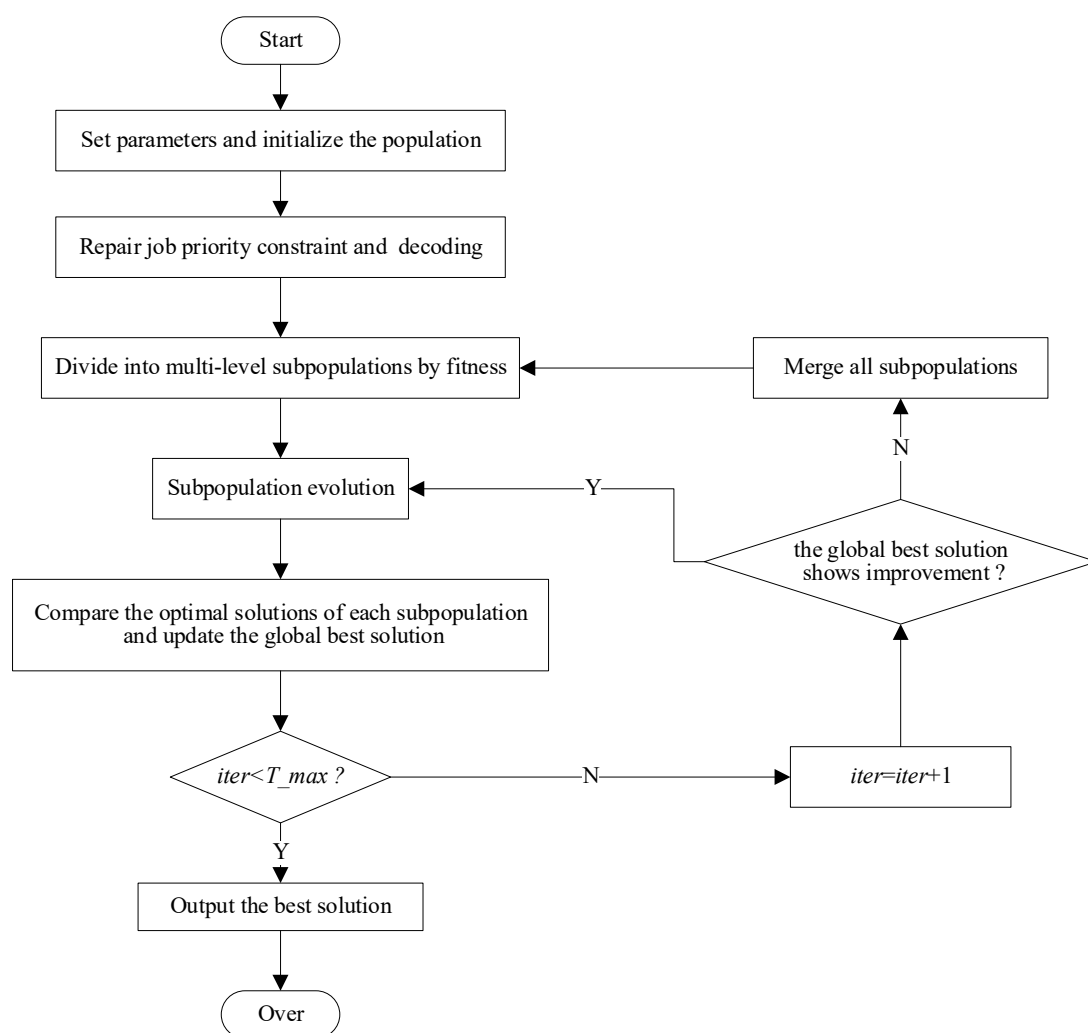


Figure 7. Flowchart of the proposed IWOA.

4.5. Computational Complexity Analysis

The time complexity per iteration of IWOA is determined by the following key procedures. Initializing the population and applying the priority-based repair mechanism to each individual's operation sequence collectively require $O(NP \cdot I \log I)$ time. Subsequently, the fitness evaluation, which involves the active scheduling decoder, is performed for each individual. The decoder schedules all I operations sequentially. For each operation, its earliest start time (EST) is computed in $O(1)$ time. Then, using EST as a lower bound, a binary search is conducted to locate the first possible start position in the sorted timeline of the assigned machine. This search step costs $O(\log K)$, where K denotes the current number of scheduled operations on that machine, averaging $O(I/m)$. Following this, a constant number of subsequent idle windows are examined on average until a feasible window is found. Therefore, scheduling a single operation has an average-case time complexity of $O(\log(I/m))$, leading to an average decoding complexity of $O(I \log(I/m))$ per individual. In the worst-case scenario, when all operations are concentrated on a single machine and their EST values are all very small, and the binary search provides limited pruning, potentially raising the cost per operation to $O(I/m)$. Consequently, decoding an individual could reach $O(I^2/m)$ in the worst case. The overall decoding complexity for the population is correspondingly $O(NP \cdot I \log(I/m))$ on average, or $O(NP \cdot I^2/m)$ in the worst case. The core position update of the Whale Optimization Algorithm involves arithmetic operations on the $2I$ -dimensional encoding vectors, incurring $O(NP \cdot I)$ time. The cross-population differential evolution strategy also requires vector operations, contributing $O(NP \cdot I)$ time per iteration. Hierarchical sub-population management, including sorting and reconstruction, is triggered only upon stagnation, with an amortized cost of $O(NP \log NP)$ per iteration.

Since fitness evaluation is executed in every iteration and dominates the computational cost, the overall average-case time complexity per iteration of IWOA is $O(NP \cdot I \log(I/m))$, while the worst-case complexity is $O(NP \cdot I^2/m)$.

5. Simulation Experiment Analysis

5.1. Instance Generation

In order to prove the effectiveness of the improved whale algorithm proposed in this paper for solving the FJSP-JPC-TST problem, a large number of experiments were carried out. Due to the lack of a standard benchmark dataset for FJSP-JPC-TST in the open literature, we constructed 12 sets of instance tests that conform to the characteristics of this problem. The rules for generating instances are shown in Table 5. Specifically, the number of jobs ranges from [10, 50], the total number of operations ranges from [30, 210], the number of operations for each job ranges from [1, 5], the number of machines ranges from [4, 9], the job priority depth is set according to the case scale, the processing time follows a discrete uniform distribution in the interval [1, 20], and the transportation time and setup time both follow a discrete uniform distribution in the interval [1, 5]. All the experiments were coded using Python 3.12 and conducted on a computer with an Intel Core i5-12450H@2.80 GHz and 8 GB RAM. In all subsequent experiments, algorithms were independently tested 10 times on each experimental instance.

Table 5. Instance settings. JN, ON, and MN denote the number of jobs, the total number of operations, and the number of machines, respectively.

Instance	JN	ON	MN	Instance	JN	ON	MN
T01	10	39	4	T07	35	140	7
T02	15	58	4	T08	40	155	7

Table 5. Cont.

Instance	JN	ON	MN	Instance	JN	ON	MN
T03	20	80	5	T09	40	170	8
T04	25	117	5	T10	45	182	8
T05	30	123	6	T11	45	201	9
T06	30	133	6	T12	50	220	9

5.2. Parameter Experiments

The proposed Improved Whale Optimization Algorithm (IWOA) primarily involves four key parameters: the population size NP , the number of subpopulations n , and the non-inertial weight coefficients $w_{\min}$ and $w_{\max}$. To determine their optimal values, a Taguchi orthogonal experimental design with four factors and four levels was employed. The parameter levels were set as follows: $NP \in \{100, 150, 200, 250\}$, $n \in \{2, 3, 4, 5\}$, $w_{\min} \in \{0.2, 0.3, 0.4, 0.5\}$, $w_{\max} \in \{0.6, 0.7, 0.8, 0.9\}$. This configuration formed an $L_{16}(4^4)$ orthogonal array, as detailed in Table 6. Using the medium-scale instance T05 as an example, ten independent experiments were conducted for each parameter combination. Each experiment was run for a maximum of 200 iterations. The global optimal solution (i.e., the minimum makespan $C_{\max}$ obtained in each experiment was recorded, and the mean makespan (denoted as “avg”) across the ten experiments was calculated for each parameter combination. Smaller mean values indicate better performance of the parameter combination. Based on the data in Table 6, the results presented in Figure 8 were obtained. The analysis indicates that the optimal parameter combination is $NP = 250$, $n = 4$, $w_{\min} = 0.3$, $w_{\max} = 0.9$.

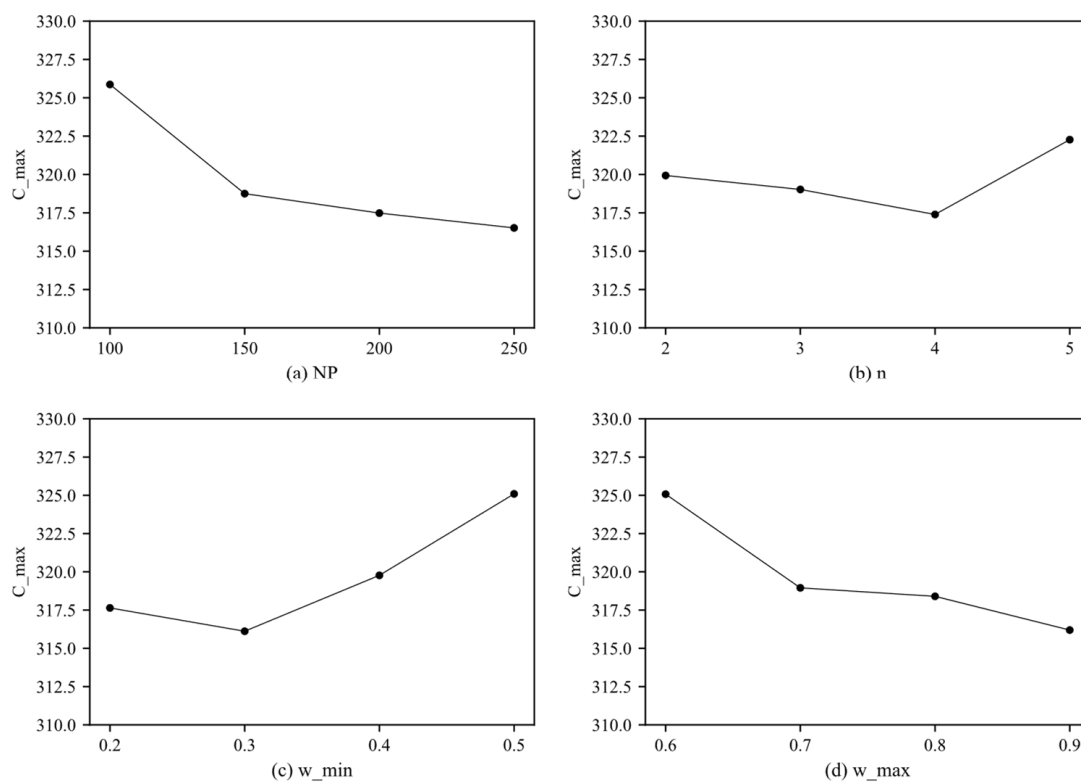


Figure 8. Influence of key algorithm parameters on the average makespan: (a) population size NP , (b) number of subpopulations n , (c) minimum inertia weight $w_{\min}$, (d) maximum inertia weight $w_{\max}$.

Table 6. Experimental Table of Parameter Combinations.

Number of Experiments	Parameter				Avg
	NP	n	$W_{\min}$	$W_{\max}$	
1	100	2	0.2	0.6	326.8
2	100	3	0.3	0.7	324.7
3	100	4	0.4	0.8	322.3
4	100	5	0.5	0.9	324.8
5	150	2	0.3	0.8	310.9
6	150	3	0.2	0.9	312.1
7	150	4	0.5	0.6	330.2
8	150	5	0.4	0.7	320.9
9	200	2	0.4	0.9	318.8
10	200	3	0.5	0.8	319.0
11	200	4	0.2	0.7	308.7
12	200	5	0.3	0.6	322.1
13	250	2	0.5	0.7	322.8
14	250	3	0.4	0.6	317.3
15	250	4	0.3	0.9	306.4
16	250	5	0.2	0.8	320.1

5.3. Ablation Experiments

To verify the effectiveness of the proposed improvement strategies, variant algorithms and ablation experiments were designed. IWOA was compared with IWOA1 (IWOA with the Multi-level sub-population optimization strategy removed), IWOA2 (IWOA with the Adaptive Inertia Weight removed), and IWOA3 (IWOA with the Cross-Population Differential Evolution Strategy removed). All algorithms were independently run 10 times, with the minimum (Best) and average (Avg) values of the objective function over multiple runs used as evaluation metrics. Due to the stochastic nature of the algorithms, Wilcoxon rank-sum tests were conducted between IWOA and the variant algorithms at a 95% confidence level. A p -value less than 0.05 indicates a significant difference between the two algorithms. In the ‘win’ column, ‘+’, ‘=’, and ‘−’ indicate that IWOA significantly outperforms, shows no significant difference from, or is significantly worse than the variant algorithm on the corresponding instance, respectively. The last row of the table counts the occurrences of these comparison symbols between IWOA and the compared algorithms, with the results shown in Table 7.

Table 7. Results of ablation experiments.

Instance	IWOA		IWOA1				IWOA2				IWOA3			
	Best	Avg	Best	Avg	p	Win	Best	Avg	p	Win	Best	Avg	p	Win
T01	145.0	145.0	145.0	145.0	1.0000	=	145.0	145.0	1.0000	=	145.0	145.0	1.0000	=
T02	205.0	207.5	208.0	210.8	0.0102	+	210.9	210.0	0.0091	+	205.0	210.8	0.0588	=
T03	199.0	204.9	206.0	209.9	0.0113	+	208.4	208.0	0.0821	=	205.0	208.1	0.0757	=
T04	292.0	296.7	303.0	306.0	0.0002	+	304.8	306.0	0.0004	+	297.0	303.2	0.0058	+
T05	301.0	304.9	306.0	309.0	0.0312	+	309.8	315.0	0.0211	+	301.0	307.2	0.1988	=
T06	306.0	313.0	317.0	324.5	0.0004	+	319.3	329.0	0.0041	+	312.0	316.9	0.1405	=
T07	318.0	328.0	330.0	341.1	0.0010	+	338.8	351.0	0.0032	+	330.0	337.1	0.0028	+
T08	347.0	358.8	361.0	373.6	0.0012	+	371.6	378.0	0.0022	+	356.0	362.8	0.2730	=
T09	354.0	363.1	388.0	402.9	0.0002	+	390.4	402.0	0.0002	+	372.0	378.8	0.0004	+
T10	341.0	347.5	358.0	365.8	0.0002	+	359.7	382.0	0.0009	+	348.0	354.9	0.0588	=
T11	326.0	338.0	342.0	351.1	0.0073	+	351.5	356.0	0.0025	+	331.0	343.6	0.1124	=
T12	483.0	495.7	514.0	523.1	0.0002	+	519.0	536.0	0.0002	+	496.0	505.8	0.0343	+
+ / = / −		11 / 1 / 0				10 / 2 / 0				4 / 8 / 0				

As shown in Table 7, IWOA achieved the smallest Best value in 10 out of 12 instances and the smallest Avg value in 11 instances, demonstrating the effectiveness of the proposed improvement strategies. The Wilcoxon test results show that the comparison outcomes between IWOA and the three variant algorithms are 11/1/0, 10/2/0, and 4/8/0, respectively, indicating that IWOA significantly outperforms the compared algorithms on the vast majority of instances. Specifically, the removal of the Multi-level sub-population optimization strategy has the most significant impact on algorithm performance. This strategy effectively maintains population diversity and avoids premature convergence through hierarchical sampling and a recombination mechanism. The removal of the Adaptive Inertia Weight strategy also leads to significant performance degradation in 10 instances, indicating that this strategy significantly improves the algorithm's balance between global exploration and local exploitation by dynamically adjusting the search step size. The impact of removing the Cross-Population Differential Evolution Strategy is relatively limited. IWOA significantly outperforms IWOA3 on the four complex instances T07, T09, and T12, while showing no significant difference on the remaining eight instances. This suggests that this strategy primarily serves as an auxiliary enhancement mechanism, providing crucial mutation when the problem structure is complex and local optima traps are deep.

5.4. Comparative Experiments

To validate the performance of the proposed Improved Whale Optimization Algorithm (IWOA), comparative experiments were conducted against other algorithms using extended benchmark instances. Given the limited number of studies specifically addressing the FJSP-JPC-TST problem, we selected FISA_RWPS [15], ABC_MSE [16], ISA_VCLDC [17], MAS_HGWO [23], and the standard WOA, which have been applied to similar scheduling problems, for comparison. To ensure fairness, the population size for all algorithms was uniformly set to 250, and the maximum number of iterations was set to 200.

The remaining parameters were set to their optimal values as reported in the original literature, specifically as follows: In FISA_RWPS, the crossover rate is 0.9 and the mutation rate is 0.3. In ABC_MSE, the search threshold is 30, and the number of scout bees is 3. In ISA_VCLDC, for each small- and medium-scale instance, the crossover rate is 0.9, and the mutation rate is 0.3; for large-scale instances, the crossover rate is 0.6, and the mutation rate is 0.01. The above algorithms are all heuristic algorithms considering job priorities, but do not consider transportation time and setup time. Therefore, the decoding method proposed in this paper is used for adjustment during the decoding stage. In MAS_HGWO, the crossover rate is 0.9. In WOA, parameter α linearly decreases from 2 to 0, the selection probability is a random number in the interval [0, 1], the spiral constant is 1, and a binary tree-based repair mechanism is further used in MAS_HGWO and WOA to ensure job priority constraints. All algorithms were independently run 10 times, and the minimum (Best), average (Avg) of the objective function over multiple runs, and the average CPU running time (Time) of the algorithms are reported, with the results shown in Table 8. Similarly, Wilcoxon rank-sum tests were conducted between IWOA and the comparison algorithms at a 95% confidence level, where a p -value less than 0.05 indicates a significant difference between the two algorithms, and the results are shown in Table 9.

Table 8. Results of comparative experiments.

Instance	IWOA			FISA_RWPS			ABC_MSE			ISA-VCLDC			MAS_HGWO			WOA		
	Best	Avg	Time	Best	Avg	Time	Best	Avg	Time	Best	Avg	Time	Best	Avg	Time	Best	Avg	Time
T01	145.0	145.0	10.3	145.0	147.1	12.8	145.0	145.5	11.7	145.0	146.3	10.7	145.0	145.2	13.5	145.0	145.9	10.4
T02	205.0	207.5	14.9	210.0	214.4	26.6	205.0	211.0	23.3	208.0	212.1	21.7	205.0	209.7	26.8	208.0	211.2	13.5

Table 8. Cont.

Instance	IWOA			FISA_RWPS			ABC_MSE			ISA-VCLDC			MAS_HGWO			WOA		
	Best	Avg	Time	Best	Avg	Time	Best	Avg	Time	Best	Avg	Time	Best	Avg	Time	Best	Avg	Time
T03	199.0	204.9	22.8	208.0	213.6	39.8	205.0	209.0	34.9	208.0	210.3	30.7	199.0	205.4	48.5	206.0	210.2	19.3
T04	292.0	296.7	33.9	306.0	310.3	73.6	297.0	301.0	57.2	301.0	305.7	38.8	296.0	301.8	76.6	302.0	306.0	31.3
T05	301.0	304.9	54.3	315.0	318.4	100.1	305.0	310.7	77.9	311.0	317.0	57.3	303.0	308.2	83.1	310.0	315.1	48.3
T06	306.0	313.0	62.3	329.0	335.7	129.2	312.0	317.8	85.4	316.0	327.5	68.8	313.0	318.4	92.7	320.0	326.4	51.9
T07	318.0	328.0	77.8	349.0	362.9	159.2	327.0	340.7	79.4	332.0	341.2	85.9	325.0	335.1	122.5	336.0	343.5	69.8
T08	347.0	358.8	107.1	378.0	386.7	170.5	358.0	365.9	142.7	364.0	374.5	118.3	352.0	363.8	138.7	368.0	376.6	102.3
T09	354.0	363.1	114.7	402.0	409.0	218.3	372.0	379.8	117.1	382.0	395.1	131.6	368.0	381.6	148.8	396.0	404.5	95.6
T10	341.0	347.5	123.1	382.0	389.2	259.1	349.0	357.0	135.2	376.0	382.5	149.3	344.0	353.8	183.9	393.0	399.2	122.6
T11	326.0	338.0	129.7	356.0	376.7	277.2	339	352.2	164.8	348.0	360.5	189.1	337.0	348.8	205.4	386.0	394.2	128.1
T12	483.0	495.7	141.3	536.0	546.9	314.4	510.0	517.2	194.7	522.0	530.9	215.0	505.0	515.6	232.5	546.0	558.8	137.8

Table 9. The Wilcoxon test results of the comparative experiment.

Instance	FISA_RWPS		ABC_MSE		ISA-VCLDC		MAS_HGWO		WOA	
	<i>p</i>	Win	<i>p</i>	Win	<i>p</i>	Win	<i>p</i>	Win	<i>p</i>	Win
T01	0.0588	=	0.4497	=	0.0588	=	1.0000	=	0.1306	=
T02	0.0012	+	0.0233	+	0.0058	+	0.1405	=	0.0065	+
T03	0.0012	+	0.0640	=	0.0046	+	0.7055	=	0.0113	+
T04	0.0002	+	0.1041	=	0.0006	+	0.0312	+	0.0002	+
T05	0.0002	+	0.0082	+	0.0004	+	0.0696	=	0.0005	+
T06	0.0002	+	0.0452	+	0.0003	+	0.0173	+	0.0002	+
T07	0.0002	+	0.0013	+	0.0003	+	0.0091	+	0.0002	+
T08	0.0002	+	0.0312	+	0.0003	+	0.1620	=	0.0002	+
T09	0.0002	+	0.0002	+	0.0002	+	0.0007	+	0.0002	+
T10	0.0002	+	0.0062	+	0.0002	+	0.0376	+	0.0002	+
T11	0.0002	+	0.0036	+	0.0002	+	0.0156	+	0.0002	+
T12	0.0002	+	0.0002	+	0.0002	+	0.0002	+	0.0002	+
+ / = / −	11/1/0		9/3/0		11/1/0		7/5/0		11/1/0	

As can be seen from Table 8, IWOA achieved the minimum Best value in 9 out of 12 cases and the minimum Avg value in all instances. The Wilcoxon test results in Table 9 show that the Avg index of the IWOA is not significantly inferior to that of the comparison algorithm in all instances, and it demonstrates a significant advantage in most instances. This indicates that the solutions obtained by IWOA are of higher quality compared to those from the other four algorithms, effectively validating the contribution of the introduced improvement strategies. The superior performance of IWOA can be attributed to several factors: First, the hierarchical sub-population optimization strategy mitigates local search stagnation through collaborative search among stratified sub-populations and a stagnation restart mechanism, thereby enhancing overall convergence performance. Second, the cross-population differential evolution strategy enriches population diversity by applying mutation perturbations to high-quality individuals. Finally, the adaptive inertia weight prioritizes global exploration during the early search stages, and shifts focus to local exploitation in later stages, improving both convergence capability and solution accuracy. In terms of runtime, IWOA demonstrates good scalability. Although its CPU time is higher than that of the standard WOA, it maintains competitive computational efficiency while obtaining better solutions. As the problem scale increases, particularly with the growth in the number of operations, the runtime of IWOA rises accordingly, which primarily stems from the computational burden of fitness evaluation. The largest test instance we used, T12, includes 50 jobs, 220 operations, and 9 machines, which represents a large-scale problem. In this instance, IWOA requires approximately 141.3 s of runtime, whereas the smaller-scale instance T01 needs only 10.3 s. This comparison indicates that although

runtime increases significantly with the total number of operations, it remains within an acceptable range. It is worth noting that the parameters of IWOA were optimized through orthogonal experiments under the current testing scale. When the problem scale increases significantly, the optimal parameter configuration may change, and the algorithm may require parameter readjustment. Overall, IWOA achieves an effective balance between computational efficiency and solution quality.

To quantify the stability of the algorithms over multiple runs, we further compared the standard deviation of the results across different algorithms, as shown in Table 10. Table 10 lists the standard deviation of the makespan obtained from 10 independent runs for each compared algorithm on 12 test instances, as well as the performance rank of IWOA in the comparative experiment (denoted by rank, where rank 1 indicates the best performance). The standard deviation directly reflects the fluctuation in solution quality when the algorithm is run multiple times; a smaller standard deviation indicates stronger robustness and repeatability of the algorithm. From Table 10, it can be seen that IWOA achieved the smallest standard deviation in 7 out of the 12 instances, demonstrating optimal stability. For a visual comparison of algorithm performance, representative test instances T02, T05, and T09 were selected for in-depth analysis. Figure 9 presents box plots comparing the results for these three instances. In terms of convergence behavior analysis, Figure 10 displays the convergence curves of the five algorithms on the three test instances. It can be observed that IWOA not only exhibits a faster initial convergence rate but also maintains a continuous improvement trend in the later iterations. The final convergence quality of IWOA is markedly superior to that of the other algorithms. Furthermore, to illustrate the practical scheduling outcomes of IWOA, Figure 11 provides feasible Gantt charts for instances T02, T05, and T09. In summary, when solving the FJSP-JPC-TST problem, compared with the comparison algorithms, the proposed IWOA demonstrates superior performance.

Table 10. Standard deviation of the comparative experiment.

Instance	IWOA	FISA_RWPS	ABC_MSE	ISA-VCLDC	MAS_HGWO	WOA	Rank
T01	0.0	2.2	1.1	1.5	0.0	1.3	1
T02	2.6	3.7	3.0	4.1	3.4	2.2	2
T03	3.7	4.2	2.6	3.2	3.7	3.6	4
T04	3.9	4.2	4.8	4.0	4.6	4.5	1
T05	4.0	4.2	5.2	4.7	4.7	4.3	1
T06	4.3	5.9	4.7	6.9	4.4	4.7	1
T07	4.8	6.0	6.3	8.7	5.3	5.3	1
T08	6.4	7.0	5.6	7.5	7.9	8.2	2
T09	6.2	5.5	5.5	8.8	11.5	6.6	3
T10	5.4	6.7	6.6	7.1	7.0	5.7	1
T11	7.6	9.4	8.7	8.6	7.9	7.2	2
T12	8.2	9.8	9.7	8.5	9.3	9.7	1

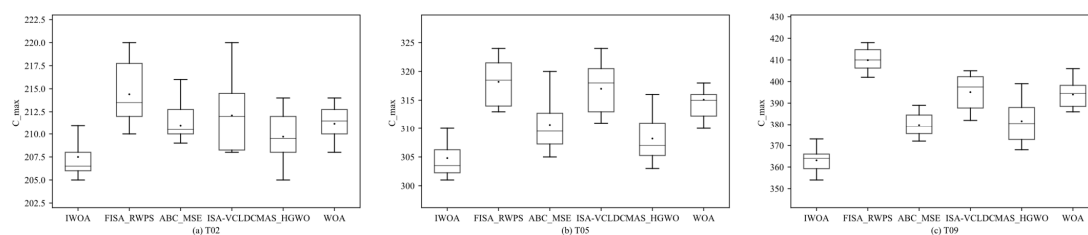
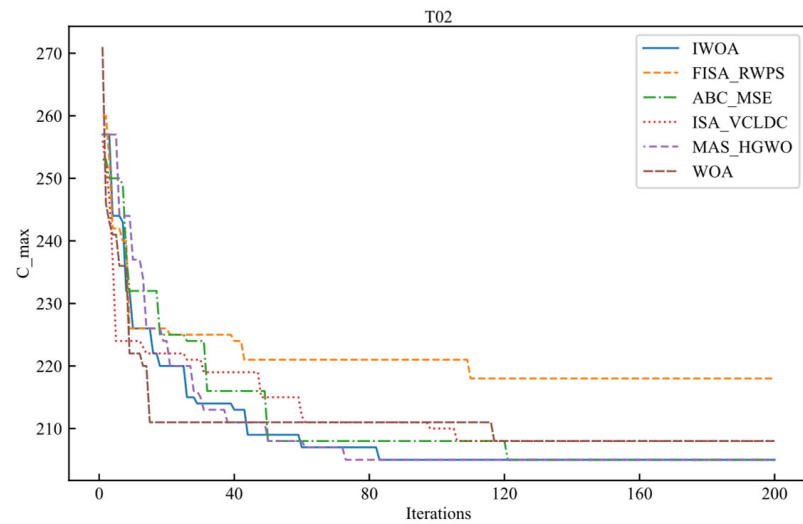
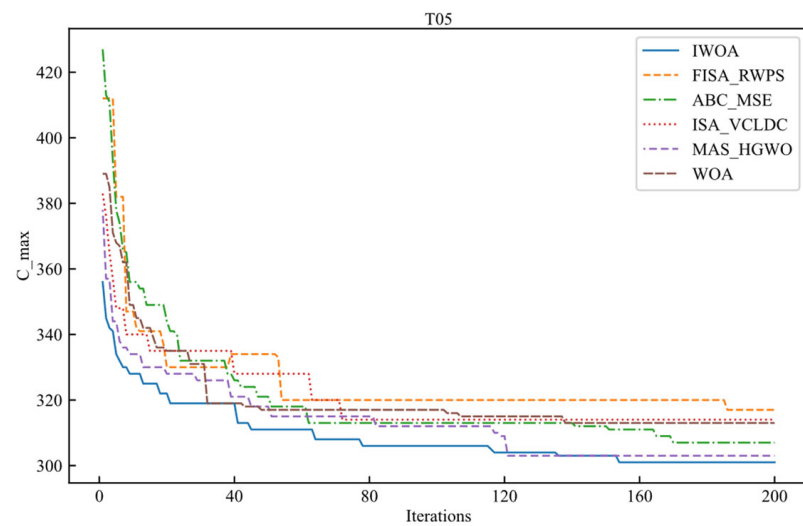


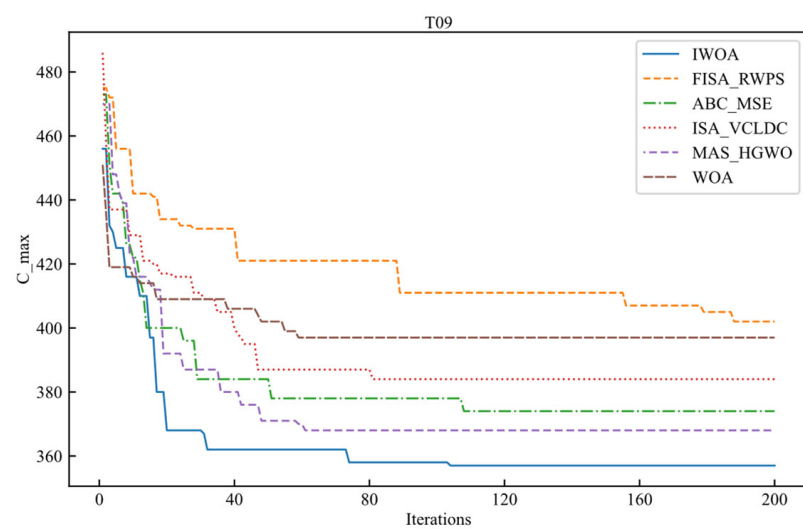
Figure 9. Box plots for different algorithms for instances (a) T02, (b) T05, and (c) T09 (The horizontal line and the dots within the box represent the median and the mean, respectively).



(a) T02

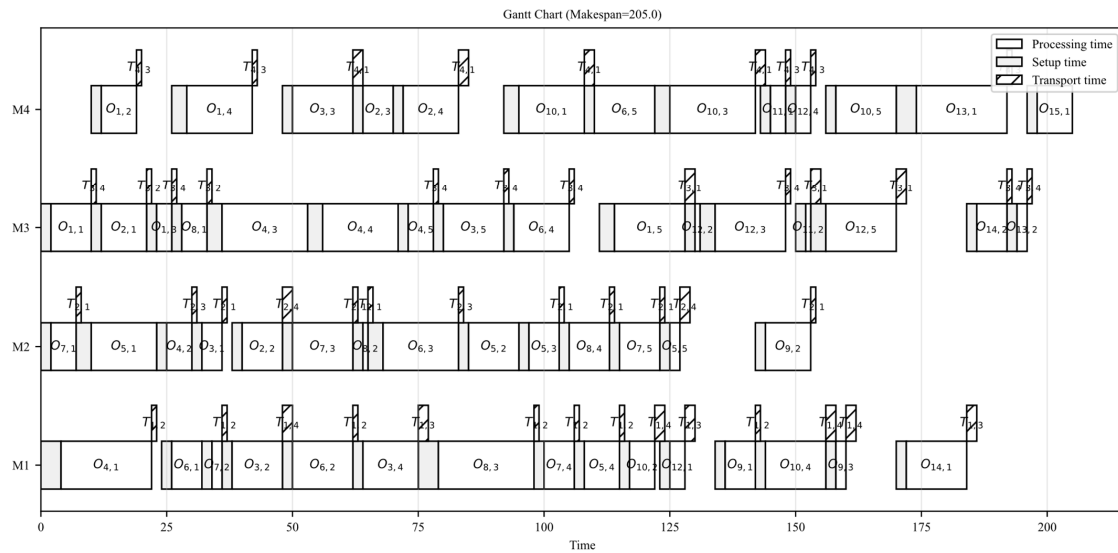


(b) T05

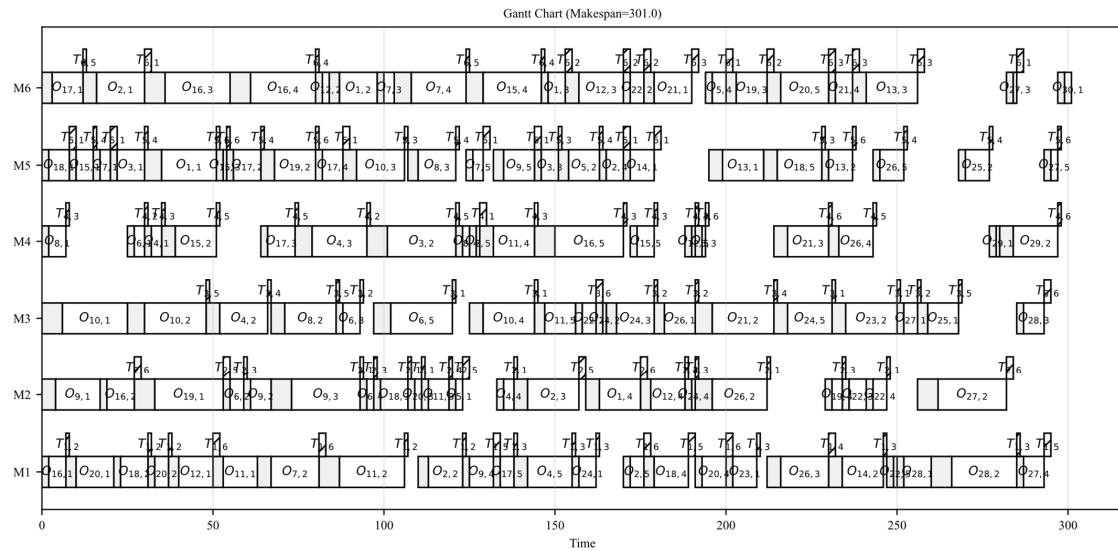


(c) T09

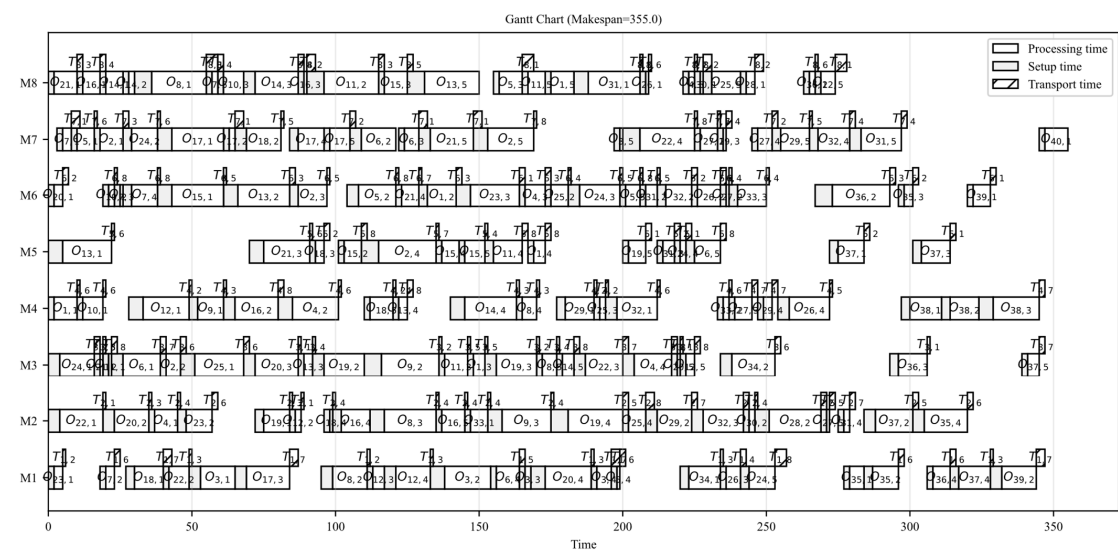
Figure 10. Convergence Curves of different algorithms for instances (a) T02, (b) T05, and (c) T09.



(a) T02



(b) T05



(c) T09

Figure 11. Gantt charts of the optimal schedules generated by the IWOA for instances (a) T02, (b) T05, and (c) T09 (The legends in Figures T05 follow the same convention as established in Figure T02).

6. Conclusions

This work addresses the flexible job-shop scheduling problem with consideration of multiple time factors and job priorities, and establishes an FJSP-JPC-TST optimization model. To solve this model, we propose an improved whale optimization algorithm (IWOA) incorporating multiple strategies. The algorithm employs a two-layer encoding scheme based on the Ranked Order Value rule, along with an encoding repair mechanism and an active scheduling decoding method, to accommodate scheduling requirements involving multiple time factors and job priorities. Furthermore, a multi-level sub-population optimization strategy is designed to prevent local search stagnation through collaborative search among stratified sub-populations and a stagnation restart mechanism. Additionally, a cross-population differential evolution strategy is introduced to enhance convergence capability and solution accuracy by applying mutation perturbations to high-quality individuals and incorporating adaptive inertia weights. Comparative experimental results demonstrate that IWOA exhibits significant advantages and reliability in solving such scheduling problems.

This research does not account for disruptions caused by dynamic events in actual job-shop production, such as machine failures and order changes, which may affect scheduling plan execution. Meanwhile, the experimental validation in this work is primarily based on simulation instances. Although we have referred to typical characteristics from the industrial scheduling literature when constructing the test cases, conclusions drawn from simulation data still require caution when directly generalizing to specific industrial production scenarios. Therefore, future work will focus on the following aspects: First, a more comprehensive modeling and analysis of dynamic events such as machine failures and urgent order insertions will be conducted to develop more robust scheduling optimization models. More importantly, we will actively seek collaboration with the industrial sector to validate and optimize this method on real manufacturing data platforms, which includes obtaining scheduling logs from actual production environments such as machining centers or assembly lines, so as to further solidify the practicality and application value of the algorithm.

Author Contributions: Conceptualization, C.Z.; Funding acquisition, Z.X.; Methodology, C.Z.; Project administration, C.Z.; Software, Z.X.; Writing—original draft, C.Z.; Writing—Review and Editing, Z.X. and C.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This work was partially supported by the National Natural Science Foundation of China (61772160, Author: Z.X.).

Data Availability Statement: The datasets GENERATED for this work can be found at <https://gitee.com/zhengchuchu0807/fjsp-jpc-tst.git> (accessed on 6 December 2025).

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

FJSP	Flexible Job-shop Scheduling Problem
BOM	Bill of Materials
FJSP-JPC-TST	flexible job-shop scheduling problem with job priority constraints, transportation time, and setup time
IWOA	Improved Whale Optimization Algorithm
ROV	Ranked Order Value

References

1. Brucker, P.; Schlie, R. Job-shop scheduling with multi-purpose machines. *Computing* **1990**, *45*, 369–375. [CrossRef]
2. Gao, K.; Cao, Z.; Zhang, L.; Chen, Z.; Han, Y.; Pan, Q. A review on swarm intelligence and evolutionary algorithms for solving flexible job shop scheduling problems. *IEEE/CAA J. Autom. Sin.* **2019**, *6*, 904–916. [CrossRef]

3. Jiang, T.; Liu, L.; Zhu, H.; Li, Y. An Improved Elephant Herding Optimization for Energy-Saving Assembly Job Shop Scheduling Problem with Transportation Times. *Axioms* **2022**, *11*, 561. [\[CrossRef\]](#)
4. Liu, L.; Jiang, T.; Zhu, H.; Shang, C. A New Interior Search Algorithm for Energy-Saving Flexible Job Shop Scheduling with Overlapping Operations and Transportation Times. *Axioms* **2022**, *11*, 306. [\[CrossRef\]](#)
5. Yin, N.; He, H.; Zhao, Y.; Chang, Y.; Wang, N. Integrating Group Setup Time Deterioration Effects and Job Processing Time Learning Effects with Group Technology in Single-Machine Green Scheduling. *Axioms* **2025**, *14*, 480. [\[CrossRef\]](#)
6. Branke, J.; Su, N.; Pickardt, C.W.; Zhang, M. Automated design of production scheduling heuristics: A review. *IEEE Trans. Evol. Comput.* **2016**, *20*, 110–124. [\[CrossRef\]](#)
7. Sun, J.; Zhang, G.; Lu, J.; Zhang, W. A hybrid many-objective evolutionary algorithm for flexible job-shop scheduling problem with transportation and setup times. *Comput. Oper. Res.* **2021**, *135*, 105263. [\[CrossRef\]](#)
8. Zhang, H.; Xu, G.; Pan, R.; Ge, H. A novel heuristic method for the energy-efficient flexible job-shop scheduling problem with sequence-dependent set-up and transportation time. *Eng. Optim.* **2022**, *54*, 1646–1667. [\[CrossRef\]](#)
9. Brandimarte, P. Routing and scheduling in a flexible job shop by tabu search. *Ann. Oper. Res.* **1993**, *41*, 157–183. [\[CrossRef\]](#)
10. Vilcot, G.; Billaut, J.C. A tabu search and a genetic algorithm for solving a bicriteria general job shop scheduling problem. *Eur. J. Oper. Res.* **2008**, *190*, 398–411. [\[CrossRef\]](#)
11. Birgin, E.G.; Feofiloff, P.; Fernandes, C.G.; de Paula, F.L.; Pedrosa, D.P.; Toffolo, T.A.M. A MILP model for an extended version of the flexible job shop problem. *Optim. Lett.* **2014**, *8*, 1417–1431. [\[CrossRef\]](#)
12. Zou, P.; Rajora, M.; Liang, S.Y. A new algorithm based on evolutionary computation for hierarchically coupled constraint optimization: Methodology and application to assembly job-shop scheduling. *J. Sched.* **2018**, *21*, 545–563. [\[CrossRef\]](#)
13. Zhu, Z.; Zhou, X.; Shao, K. A novel approach based on Neo4j for multi-constrained flexible job shop scheduling problem. *Comput. Ind. Eng.* **2019**, *130*, 671–686. [\[CrossRef\]](#)
14. Zhu, Z.; Zhou, X. Flexible job-shop scheduling problem with job precedence constraints and interval grey processing time. *Comput. Ind. Eng.* **2020**, *148*, 106781. [\[CrossRef\]](#)
15. Gao, Y.; Xie, Z.; Yang, D.; Yu, X. Flexible integrated scheduling algorithm based on remaining work probability selection coding. *Expert Syst.* **2021**, *38*, e12690. [\[CrossRef\]](#)
16. Xie, Z.; Yang, D.; Ma, M.; Yu, X. An improved artificial bee colony algorithm for the flexible integrated scheduling problem using networked devices collaboration. *Int. J. Coop. Inf. Syst.* **2020**, *29*, 2050015. [\[CrossRef\]](#)
17. Lei, Q.; Guo, W.; Song, Y. Integrated scheduling algorithm based on an operation relationship matrix table for tree-structured products. *Int. J. Prod. Res.* **2018**, *56*, 5437–5456. [\[CrossRef\]](#)
18. Zhu, Z.; Zhou, X.; Cao, D.; Li, M. A shuffled cellular evolutionary grey wolf optimizer for flexible job shop scheduling problem with tree-structure job precedence constraints. *Appl. Soft Comput.* **2022**, *125*, 109235. [\[CrossRef\]](#)
19. Li, H.; Wang, X.; Peng, J. A hybrid differential evolution algorithm for flexible job shop scheduling with outsourcing operations and job priority constraints. *Expert Syst. Appl.* **2022**, *201*, 117182. [\[CrossRef\]](#)
20. Zhang, G.; Sun, J.; Lu, X.; Zhang, H. An improved memetic algorithm for the flexible job shop scheduling problem with transportation times. *Meas. Control* **2020**, *53*, 1518–1528. [\[CrossRef\]](#)
21. Homayouni, S.M.; Fontes, D.B.M.M. Production and transport scheduling in flexible job shop manufacturing systems. *J. Glob. Optim.* **2021**, *79*, 437–465. [\[CrossRef\]](#)
22. Jiang, T.; Zhu, H.; Liu, L.; Gong, Q. Energy-conscious flexible job shop scheduling problem considering transportation time and deterioration effect simultaneously. *Sustain. Comput. Inform. Syst.* **2022**, *35*, 100680. [\[CrossRef\]](#)
23. Pal, M.; Mittal, M.L.; Soni, G.; Chouhan, S.S.; Kumar, M. A multi-agent system for FJSP with setup and transportation times. *Expert Syst. Appl.* **2023**, *216*, 119474. [\[CrossRef\]](#)
24. Chen, X.; Li, J.; Wang, Z.; Li, J.; Gao, K. A genetic programming based cooperative evolutionary algorithm for flexible job shop with crane transportation and setup times. *Appl. Soft Comput.* **2025**, *169*, 111456. [\[CrossRef\]](#)
25. He, Z.; Tang, B.; Gu, C.; Ran, N. Enhanced Monte-Carlo tree search for dynamic flexible job shop scheduling with transportation time constraints. *Expert Syst.* **2025**, *42*, e13727. [\[CrossRef\]](#)
26. Meng, L.; Gao, K.; Ren, Y.; Zhang, B.; Sang, H.; Zhang, C. Novel MILP and CP models for distributed hybrid flowshop scheduling problem with sequence-dependent setup times. *Swarm Evol. Comput.* **2022**, *71*, 101063. [\[CrossRef\]](#)
27. Li, M.; Lei, D. An imperialist competitive algorithm with feedback for energy-efficient flexible job shop scheduling with transportation and sequence-dependent setup times. *Eng. Appl. Artif. Intell.* **2021**, *103*, 104320. [\[CrossRef\]](#)
28. Sun, H.-B.; Chen, Y.-R.; Lin, J. An effective two-stage heuristic for scheduling the distributed assembly flowshops with sequence dependent setup times. *Comput. Oper. Res.* **2025**, *173*, 106890. [\[CrossRef\]](#)
29. Karabulut, K.; Ztop, H.; Kizilay, D.; Tasgetiren, M.F.; Kandiller, L. An evolution strategy approach for the distributed permutation flowshop scheduling problem with sequence-dependent setup times. *Comput. Oper. Res.* **2022**, *142*, 105733. [\[CrossRef\]](#)
30. Chen, X.; Li, J.; Wang, Z.; Chen, Q.; Gao, K.; Pan, Q. Optimizing Dynamic Flexible Job Shop Scheduling Using an Evolutionary Multi-Task Optimization Framework and Genetic Programming. *IEEE Trans. Evol. Comput.* **2025**, *29*, 1502–1516. [\[CrossRef\]](#)

31. Liu, X.; Jiang, D.; Tao, B.; Jiang, G.Z.; Sun, Y.; Kong, J.Y.; Chen, B.J. Genetic algorithm-based trajectory optimization for digital twin robots. *Front. Bioeng. Biotechnol.* **2021**, *9*, 793782. [[CrossRef](#)] [[PubMed](#)]
32. Teng, G. An improved genetic algorithm for dual-resource constrained flexible job shop scheduling problem with tool-switching dependent setup time. *Expert Syst. Appl.* **2025**, *281*, 123456. [[CrossRef](#)]
33. Kong, J.; Wang, Z. Research on flexible job shop scheduling problem with handling and setup time based on improved discrete particle swarm algorithm. *Appl. Sci.* **2024**, *14*, 2586. [[CrossRef](#)]
34. Shi, K.J.; Huang, L.; Jiang, D.; Sun, Y.; Tong, X.L.; Xie, Y.M.; Fang, Z.F. Path planning optimization of intelligent vehicle based on improved genetic and ant colony hybrid algorithm. *Front. Bioeng. Biotechnol.* **2022**, *10*, 905983. [[CrossRef](#)]
35. Jasim, A.M.; Jasim, B.H.; Bures, V. A novel grid-connected microgrid energy management system with optimal sizing using hybrid grey wolf and cuckoo search optimization algorithm. *Front. Energy Res.* **2022**, *10*, 960141. [[CrossRef](#)]
36. Chen, X.L.; Li, J.Q.; Xu, Y. Q-learning based multi-objective immune algorithm for fuzzy flexible job shop scheduling problem considering dynamic disruptions. *Swarm Evol. Comput.* **2023**, *83*, 101398. [[CrossRef](#)]
37. Lv, Z.; Zhao, Y.; Kang, H.; Gao, Z.; Qin, Y. An improved Harris hawk optimization algorithm for flexible job shop scheduling problem. *Comput. Mater. Contin.* **2024**, *78*, 2337–2360. [[CrossRef](#)]
38. Luo, Q.; Deng, Q.; Gong, G.; Guo, X.; Liu, X. A distributed flexible job shop scheduling problem considering worker arrangement using an improved memetic algorithm. *Expert Syst. Appl.* **2024**, *239*, 117984. [[CrossRef](#)]
39. Dehghani, M.; Trojovský, P. Osprey optimization algorithm: A new bio-inspired metaheuristic algorithm for solving engineering optimization problems. *Sci. Rep.* **2023**, *13*, 19583. [[CrossRef](#)]
40. Zhang, X.F.; Xiao, F.; Tong, X.L.; Yun, J.T.; Liu, Y.; Sun, Y.; Chen, B.J. Time optimal trajectory planning based on improved sparrow search algorithm. *Front. Bioeng. Biotechnol.* **2022**, *10*, 852408. [[CrossRef](#)]
41. Mirjalili, S.; Lewis, A. The whale optimization algorithm. *Adv. Eng. Softw.* **2016**, *95*, 51–67. [[CrossRef](#)]
42. Got, A.; Moussaoui, A.; Zouache, D. Hybrid filter-wrapper feature selection using whale optimization algorithm: A multi-objective approach. *Expert Syst. Appl.* **2021**, *183*, 115312. [[CrossRef](#)]
43. Aziz, M.A.E.; Ewees, A.A.; Hassanien, A.E. Multi-objective whale optimization algorithm for content-based image retrieval. *Multimed. Tools Appl.* **2018**, *77*, 27659–27679. [[CrossRef](#)]
44. Tang, R.; Tang, X.; Zhao, H. Enhancing whale optimization algorithm with differential evolution and Lévy flight for robot path planning. *Int. J. Adv. Comput. Sci. Appl.* **2024**, *15*, 401. [[CrossRef](#)]
45. Gurusamy, B.M.; Rangarajan, P.K.; Altalbe, A. Whale-optimized LSTM networks for enhanced automatic text summarization. *Front. Artif. Intell.* **2024**, *7*, 1399168. [[CrossRef](#)] [[PubMed](#)]
46. Got, A.; Moussaoui, A.; Zouache, D. A guided population archive whale optimization algorithm for solving multiobjective optimization problems. *Expert Syst. Appl.* **2020**, *141*, 112972. [[CrossRef](#)]
47. Feng, S.; Jia, J. Acceleration sensor placement technique for vibration test in structural health monitoring using microhabitat frog-leaping algorithm. *Struct. Health Monit.* **2017**, *16*, 589–602. [[CrossRef](#)]
48. Yi, W.; Chen, Y.; Pei, Z.; Lu, J. Adaptive differential evolution with ensembling operators for continuous optimization problems. *Swarm Evol. Comput.* **2022**, *69*, 100994. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.